

Collaborative Protection Profile for Application Software



Version: 2.0

2026-07-21

Application Software International Technical Community (AppSW-iTC)

Revision History

Version	Date	Comment
v 1.0	2014-10-20	Initial release
v 1.1	2014-11-05	Addition to TLS cipher suite selections
v 1.2	2016-04-22	Added server-side TLS requirements (selection-based) Multiple clarification based on NIAP TRRT inquiries Refactored FDP_DEC_EXT.1 into separate components
v 1.3	2019-03-01	Incorporated available Technical Decisions Refactored FPT_TUD Added a selection to FPT_DIT Moved SWID Tags requirement Leveraged TLS Package Added equivalency section
v 1.4	2021-10-07	Incorporated applicable Technical Decisions Updated to TLS FP 2.1 Incorporated SSH FP 2.0
v 2.0	2025-06-16	Rebaselined from the NIAP Protection Profile for Application Software, Version 2.0, as the source baseline rather than revising the prior collaborative Protection Profile for Application Software Version 1.4 Converted to collaborative Protection Profile maintained by the Application Software International Technical Community (AppSW-iTC) CC :2022 conversion Updating for TLS FP, SSH FP, and X.509 FP TDs and GitHub Issues CNSA 2.0 updates ALC FLR Updates
v 2.0 Draft	2026-07-21	Draft-review updates.

Contents

- 1 Introduction
 - 1.1 Overview
 - 1.2 Terms
 - 1.2.1 Common Criteria Terms
 - 1.2.2 Technical Terms
 - 1.3 Compliant Targets of Evaluation
 - 1.3.1 TOE Boundary
 - 1.3.2 Managed Runtime and Application Framework Dependencies
 - 1.4 Use Cases
 - 1.5 Distributed and Microservices TOE Architectures
 - 1.6 Platforms with Specific EAs
- 2 Conformance Claims
- 3 Security Problem Definition
 - 3.1 Threats
 - 3.2 Assumptions
 - 3.3 Organizational Security Policies
- 4 Security Objectives
 - 4.1 Security Objectives for the Operational Environment
 - 4.2 Security Objectives Rationale
- 5 Security Requirements
 - 5.1 Security Functional Requirements

- 5.1.1 Cryptographic Support (FCS)
- 5.1.2 User Data Protection (FDP)
- 5.1.3 Security Management (FMT)
- 5.1.4 Privacy (FPR)
- 5.1.5 Protection of the TSF (FPT)
- 5.1.6 Trusted Path/Channels (FTP)
- 5.1.7 TOE Security Functional Requirements Rationale
- 5.2 Security Assurance Requirements
 - 5.2.1 Class ASE: Security Target
 - 5.2.2 Class ADV: Development
 - 5.2.3 Class AGD: Guidance Documentation
 - 5.2.4 Class ALC: Life-cycle Support
 - 5.2.5 Class ATE: Tests
 - 5.2.6 Class AVA: Vulnerability Assessment
- Appendix A - Optional Requirements
 - A.1 Strictly Optional Requirements
 - A.1.1 Class ALC: Life-cycle Support
 - A.2 Objective Requirements
 - A.2.1 Protection of the TSF (FPT)
 - A.3 Implementation-dependent Requirements
- Appendix B - Selection-based Requirements
 - B.1 Cryptographic Support (FCS)
 - B.2 Protection of the TSF (FPT)
- Appendix C - Extended Component Definitions
 - C.1 Extended Components Table
 - C.2 Extended Component Definitions
 - C.2.1 Cryptographic Support (FCS)
 - C.2.1.1 FCS_CKM_EXT Cryptographic Key Management
 - C.2.1.2 FCS_HTTPS_EXT HTTPS Protocol
 - C.2.1.3 FCS_PBKDF_EXT Password Conditioning
 - C.2.1.4 FCS_RBG_EXT Random Bit Generation
 - C.2.1.5 FCS_STO_EXT Storage of Credentials
 - C.2.2 Privacy (FPR)
 - C.2.2.1 FPR_ANO_EXT User Consent for Transmission of Personally Identifiable Information
 - C.2.3 Protection of the TSF (FPT)
 - C.2.3.1 FPT_AEX_EXT Anti-Exploitation Capabilities
 - C.2.3.2 FPT_API_EXT Use of Supported Services and APIs
 - C.2.3.3 FPT_IPC_EXT Protection of Non-Network Local Inter-Process Communication
 - C.2.3.4 FPT_IDV_EXT Software Identification and Versions
 - C.2.3.5 FPT_LIB_EXT TSF Use of Third Party Libraries
 - C.2.3.6 FPT_TUD_EXT Trusted Updates
 - C.2.4 Security Management (FMT)
 - C.2.4.1 FMT_CFG_EXT Secure by Default Configuration
 - C.2.4.2 FMT_MEC_EXT Supported Configuration Mechanism
 - C.2.5 Trusted Path/Channels (FTP)
 - C.2.5.1 FTP_DIT_EXT Protection of Data in Transit
 - C.2.6 User Data Protection (FDP)
 - C.2.6.1 FDP_DAR_EXT Data-at-Rest Encryption
 - C.2.6.2 FDP_DEC_EXT Access to Platform Resources
 - C.2.6.3 FDP_NET_EXT Network Communications
- Appendix D - Entropy Documentation and Assessment
 - D.1 Design Description
 - D.2 Entropy Justification
 - D.3 Operating Conditions
 - D.4 Health Testing
- Appendix E - Application Software Equivalency Guidelines
 - E.1 Introduction
 - E.2 Approach to Equivalency Analysis
 - E.3 Specific Guidance for Determining Product Model Equivalence
 - E.4 Specific Guidance for Determining Product Version Equivalence
 - E.5 Specific Guidance for Determining Platform Equivalence
 - E.5.1 Platform Equivalence—Hardware/Virtual Hardware Platforms
 - E.5.2 Platform Equivalence—OS Platforms
 - E.5.3 Software-based Execution Environment Platform Equivalence
 - E.6 Level of Specificity for Tested Configurations and Claimed Equivalent Configurations
- Appendix F - Component Implementation Equivalence Within a Distributed TOE

- F.1 Purpose and Relationship to Product Equivalence
- F.2 Factors for Establishing a Component Implementation-Equivalence Class
- F.3 Representative Execution and Complete Coverage
- Appendix G - Acronyms
- Appendix H - Bibliography

1 Introduction

1.1 Overview

The scope of this Protection Profile (PP) is to describe the security functionality of application software in terms of [CC] and to define functional and assurance requirements for such software. In recent years, software attacks have shifted from targeting operating systems to targeting applications. This has been the natural response to improvements in operating system security and development processes. As a result, it is paramount that the security of applications be improved to reduce the risk of compromise.

1.2 Terms

The following sections list Common Criteria and technology terms used in this document.

1.2.1 Common Criteria Terms

Assurance	Grounds for confidence that a TOE meets the SFRs [CC].
Base Protection Profile (Base-PP)	Protection Profile used as a basis to build a PP-Configuration.
Collaborative Protection Profile (cPP)	A Protection Profile developed by international technical communities and approved by multiple schemes.
Common Criteria (CC)	Common Criteria for Information Technology Security Evaluation (International Standard ISO/IEC 15408).
Common Criteria Testing Laboratory	Within the context of the Common Criteria Evaluation and Validation Scheme (CCEVS), an IT security evaluation facility accredited by the National Voluntary Laboratory Accreditation Program (NVLAP) and approved by the NIAP Validation Body to conduct Common Criteria-based evaluations.
Common Evaluation Methodology (CEM)	Common Evaluation Methodology for Information Technology Security Evaluation.
Direct Rationale	A type of Protection Profile, PP-Module, or Security Target in which the security problem definition (SPD) elements are mapped directly to the SFRs and possibly to the security objectives for the operational environment. There are no security objectives for the TOE.
Distributed TOE	A TOE composed of multiple components operating as a logical whole.
Extended Package (EP)	A deprecated document form for collecting SFRs that implement a particular protocol, technology, or functionality. See Functional Packages.
Functional Package (FP)	A document that collects SFRs for a particular protocol, technology, or functionality.
Operational Environment (OE)	Hardware and software that are outside the TOE boundary that support the TOE functionality and security policy.
Protection Profile (PP)	An implementation-independent set of security requirements for a category of products.
Protection Profile Configuration (PP-Configuration)	A comprehensive set of security requirements for a product type that consists of at least one Base-PP and at least one PP-Module.
Protection Profile Module (PP-Module)	An implementation-independent statement of security needs for a TOE type complementary to one or more Base-PPs.
Security Assurance Requirement (SAR)	A requirement to assure the security of the TOE.

Security Functional Requirement (SFR)	A requirement for security enforcement by the TOE.
Security Target (ST)	A set of implementation-dependent security requirements for a specific product.
Target of Evaluation (TOE)	The product under evaluation.
TOE Security Functionality (TSF)	The security functionality of the product under evaluation.
TOE Summary Specification (TSS)	A description of how a TOE satisfies the SFRs in an ST.

1.2.2 Technical Terms

Address Space Layout Randomization (ASLR)	An anti-exploitation feature which loads memory mappings into unpredictable locations. ASLR makes it more difficult for an attacker to redirect control to code that they have introduced into the address space of an application process.
Application (app)	Software that runs on a platform and performs tasks on behalf of the user or owner of the platform, as well as its supporting documentation. For a single-component TOE, the terms <i>TOE</i> and <i>application</i> are interchangeable in this document. For a distributed TOE, the TOE is the collective boundary and an application component is an identified TOE Component within that boundary.
Application Framework	A software framework used by an application to provide application structure or security-relevant services such as request routing, authentication, authorization, session management, object binding, serialization or deserialization, configuration management, cryptographic provider selection, management endpoint exposure, or other application behavior. Examples include web, enterprise, dependency injection, remote procedure call, and service frameworks.
Application Programming Interface (API)	A specification of routines, data structures, object classes, and variables that allows an application to make use of services provided by another software component, such as a library. APIs are often provided for a set of libraries included with the platform.
Communication Relationship Type	A class of communications between an identified initiating TOE Component and an identified receiving TOE Component, which may be the same TOE Component, where the communications have the same security purpose, interfaces, protocol roles, protection mechanism, identity and trust model, and security-relevant configuration. Relationships are separate types when a difference can affect an applicable SFR or Evaluation Activity.
Component Implementation-Equivalence Class	For one or more explicitly identified SFR iterations and implementation-specific portions of Evaluation Activities, a set of named SFR Implementation Instances, used by one or more TOE Components in identified security-relevant configurations where applicable, that have the same security-relevant implementation and version, wrapper or integration behavior, security-relevant build options and implementation configuration relevant to the reused activities, role and exercised code path, cryptographic provider where applicable, and immediate platform services and interfaces relevant to the reused activities. Peripheral differences may exist only when they are identified and demonstrated not to affect the scoped Evaluation Activity results. Channel-specific deployed values that are covered by retained interface or channel checks do not, by themselves, prevent class membership. Interfaces and communication relationships are coverage targets rather than class members.
Credential	Data that establishes the identity of a user, e.g. a cryptographic key or password.
Data Execution Prevention (DEP)	An anti-exploitation feature of modern operating systems executing on modern computer hardware, which enforces a non-execute permission on pages of memory. DEP prevents pages of memory from containing both data and instructions, which makes it more difficult for an attacker to introduce and execute code.
Developer	An entity that writes application software. For the purposes of this document, vendors and developers are the same.
Managed Runtime	A software execution environment, other than the operating system, that interprets, compiles, loads, hosts, or otherwise mediates execution of application code. Examples include virtual machines, language runtimes, managed execution environments, embedded application servers, and scripting runtimes.

Mobile Code	Software transmitted from a remote system for execution within a limited execution environment on the local system. Typically, there is no persistent installation and execution begins without the user's consent or even notification. Examples of mobile code technologies include JavaScript, Java applets, Adobe Flash, and Microsoft Silverlight.
Operating System (OS)	Software that manages hardware resources and provides services for applications.
Personally Identifiable Information (PII)	Any information about an individual maintained by an agency, including, but not limited to, education, financial transactions, medical history, and criminal or employment history and information which can be used to distinguish or trace an individual's identity, such as their name, social security number, date and place of birth, mother's maiden name, biometric records, etc., including any other personal information which is linked or linkable to an individual. [OMB]
Platform	The environment in which application software runs. The platform can be an operating system, hardware environment, a software based execution environment, or some combination of these. These types of platforms may also run atop other platforms.
Protected Channel Instance	A materially distinct realization of a Communication Relationship Type. A channel instance is distinguished by an interface, protocol role, credential or trust configuration, endpoint-identity processing, protection configuration, or interruption and recovery behavior that can affect an applicable Evaluation Activity. Otherwise equivalent channels that differ only because additional Runtime Replicas create them are not separate instances unless an Evaluation Activity expressly requires every deployed instance to be examined or tested.
Running-Payload Activity	The portion of an Evaluation Activity whose evaluation object is a TOE application payload executing in a Linux container and whose procedure examines or observes its Linux userspace process, executable, effective filesystem, or effective application configuration. The term does not include activity portions whose evaluation object is an image or update artifact, installation or update workflow, container runtime or orchestrator configuration, or external interface or channel.
Runtime Replica	A TOE Component Instance that has the same application artifact, security-relevant configuration, role, claimed SFR behavior, and relevant platform interface as another deployed instance of that TOE Component. Instance-specific identities, addresses, credentials, or similar deployed values may differ when the difference does not alter the claimed SFR behavior or an Evaluation Activity result and remains covered by the required instance-, interface-, or channel-specific checks. Horizontal scaling through Runtime Replicas does not create an additional TOE Component. An instance with a different application artifact is not a Runtime Replica and shall be represented as a separately identified TOE Component or explicitly identified configuration. An instance with another difference that can affect an SFR or Evaluation Activity is also not a Runtime Replica and shall be represented separately in the same manner.
SFR Implementation Instance	A unique use of a security-relevant implementation by a TOE Component, in an identified security-relevant configuration where applicable, to satisfy one or more SFRs. Additional Runtime Replicas that deploy that use do not create additional SFR Implementation Instances.
Sensitive Data	Sensitive data may include all user or enterprise data or may be specific application data such as emails, messaging, documents, calendar items, and contacts. Sensitive data must minimally include PII, credentials, and keys. Sensitive data shall be identified in the application's TSS by the ST author.
Stack Cookie	An anti-exploitation feature that places a value on the stack at the start of a function call, and checks that the value is the same at the end of the function call. This is also referred to as Stack Guard, or Stack Canaries.
TOE Component	A named, logical, separately deployable portion of the TOE that is identified in the ST and mapped to the base cPP, applicable PP-Modules, and relevant SFRs. A TOE Component may have one or more deployed TOE Component Instances. A different application artifact, or a difference in security-relevant configuration, role, claimed SFR behavior, or relevant platform interface that can affect an SFR or Evaluation Activity, requires a separately identified TOE Component or explicitly identified configuration.
TOE Component Instance	A deployed occurrence of a TOE Component.
Vendor	An entity that sells application software. For purposes of this document, vendors and developers are the same. Vendors are responsible for maintaining and updating application software.

1.3 Compliant Targets of Evaluation

The requirements in this document apply to application software which runs on any type of platform. Some application types are covered by more specific **PPs**, which may be expressed as **PP-Modules** of this **PP**. Such applications are subject to the requirements of both this **PP** and the **PP-Module** that addresses their special functionality. **PPs** for some particularly specialized applications may not be expressed as **PP-Modules** at this time, though the requirements in this document should be seen as objectives for those highly specialized applications.

Although the requirements in this document apply to a wide range of application software, consult guidance from the relevant national schemes to determine when formal Common Criteria evaluation is expected for a particular type of application. This may vary depending upon the nature of the security functionality of the application.

1.3.1 TOE Boundary

The **TOE** consists of the vendor-provided application component or components identified in the **ST**. It may be installed or deployed on one or more platforms and may execute as a single application or as multiple cooperating components. A platform may be an operating system (Figure 1), hardware environment, software-based execution environment, or some combination of these (Figure 2). A platform may itself operate within a virtualized or layered environment that abstracts underlying hardware or platform services from the **TOE**. Functionality provided by an abstracted layer is part of the operational environment unless the **ST** explicitly includes that layer in the **TOE**. The **ST** shall identify any operational environment service upon which the **TOE** relies to satisfy a security requirement and the configuration required of that service.

Some evaluation activities are specific to the platform on which an application component runs in order to provide precision and repeatability. The platforms currently recognized by this **CPP** are those specified in the **SFR** Evaluation Activities. To test on a platform for which there are no applicable evaluation activities, an interested party may contact the AppSW-iTC with proposed activities. The AppSW-iTC will determine whether the proposed platform is appropriate for the **CPP** and accept, reject, or develop evaluation activities as necessary in coordination with the technical community.

This **CPP** applies to application software regardless of its function, delivery mechanism, or deployment topology. Application software may provide an interactive client, a background or server-side service, or a set of containerized or otherwise distributed application payloads. It may be delivered as a native or mobile package, container image, deployment bundle, or other application artifact. Delivery, packaging, and deployment topology do not by themselves determine the **TOE** boundary.

The **TOE** boundary shall include every vendor-provided software component that implements or supports a claimed **SFR**, as well as any additional vendor-provided components explicitly included in the **ST**. Such components may include user-space processes, libraries, services, sidecars, drivers, and other platform extensions. Platform-provided firmware, operating-system kernels and drivers, runtimes, orchestrators, and other system services are part of the operational environment unless explicitly included in the **TOE**. Co-packaging or co-deployment with the application does not by itself change that classification. Additional requirements for **TOEs** comprising separately deployed components are defined in Section 1.5 Distributed and Microservices TOE Architectures.

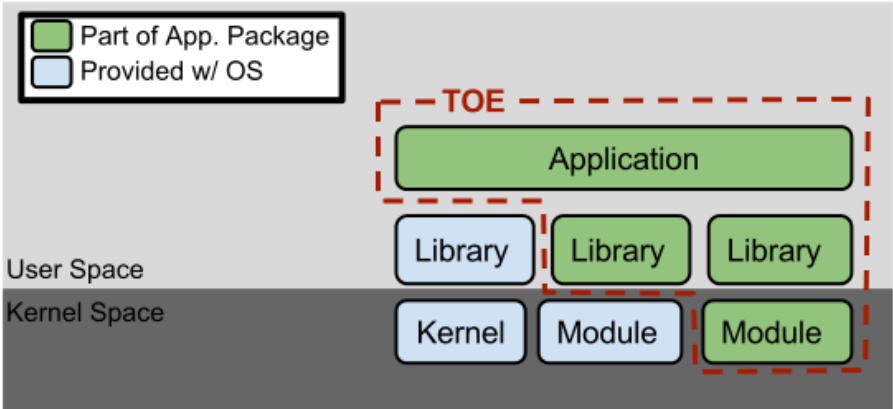


Figure 1: **TOE** as an Application and Kernel Module Running on an Operating System

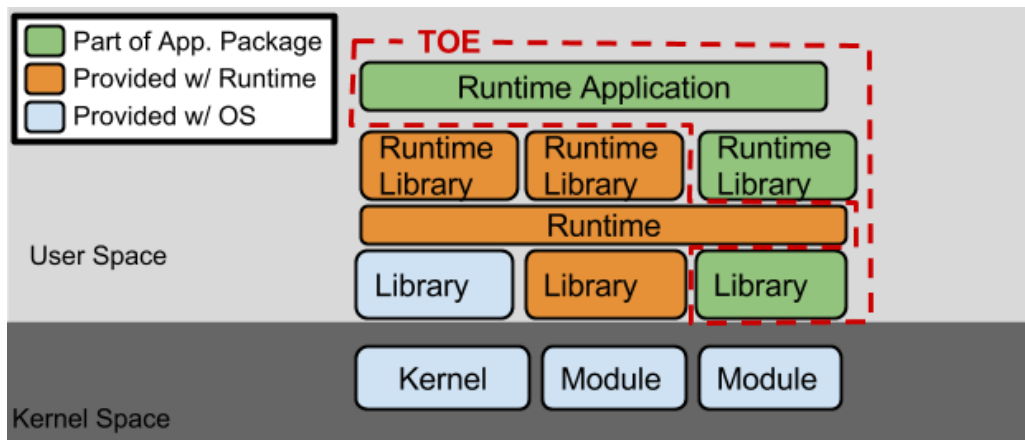


Figure 2: TOE as an Application Running in an Execution Environment Plus Native Code

1.3.2 Managed Runtime and Application Framework Dependencies

Some applications rely on managed runtimes or application frameworks to execute application code or to provide security-relevant services. A managed runtime or application framework is not automatically considered a platform equivalent to an operating system. The ST shall identify whether each security-relevant managed runtime or application framework is included in the TOE, bundled with the TOE, provided by the operational environment, or relied upon as a platform or operational environment dependency.

If a managed runtime or application framework is included in the TOE boundary, the ST shall identify it and address its security-relevant configuration, update mechanism, invoked APIs, included libraries, and any security-relevant services it provides to the application. If a runtime or framework is delivered or deployed with the TOE but remains outside the TOE boundary, or is otherwise provided by the operational environment, the ST shall identify the dependency and the guidance shall describe the environmental configuration required for the TOE's claimed security functionality.

Security-relevant managed runtime or application framework behavior includes, but is not limited to, code loading, classpath or module path processing, just-in-time compilation, native interface exposure, reflection, object serialization or deserialization, object binding, routing, authentication, authorization, session management, management or debug interfaces, management endpoints, cryptographic provider selection, trust store behavior, configuration loading, and interpretation of environment variables or external configuration sources. The evaluator shall assess these behaviors through the applicable SFR assurance activities rather than treating the runtime or framework as opaque implementation detail.

1.4 Use Cases

Requirements in this PP are designed to address the security problem in the following use cases. These use cases are intentionally very broad, as many specific use cases exist for application software. Many applications may be used in combinations of these broad use cases, and evaluation against PP-Modules of this PP, when available, may be most appropriate for some application types.

[USE CASE 1] Content Creation

The application allows a user to create content, saving it to either local or remote storage. Example content includes text documents, presentations, and images.

[USE CASE 2] Content Consumption

The application allows a user to consume content, retrieving it from either local or remote storage. Example content includes web pages and video.

[USE CASE 3] Communication

The application allows for communication interactively or non-interactively with other users or applications over a communications channel. Example communications include instant messages, email, and voice.

1.5 Distributed and Microservices TOE Architectures

A TOE may consist of multiple TOE Components that collectively provide the TOE security functionality. A TOE Component is a named, logical, separately deployable portion of the TOE that is identified in the ST and mapped to the base CPP, applicable PP-Modules, and relevant SFRs. A TOE Component Instance is a deployed occurrence of a TOE Component. Examples include server-agent products, clustered application deployments, and microservices-based applications composed of multiple application payloads. If a TOE is deployed across multiple TOE Component Instances, the ST shall identify each TOE Component and its permitted instance or scaling topology, including any Runtime Replicas; describe the role of each TOE Component; identify which TOE Components implement each claimed SFR or SFR element; describe every Communication Relationship Type and Protected Channel Instance required by the applicable Evaluation Activities; and distinguish TOE Components and their instances from operational environment components.

Illustrative distributed TOE boundary and evaluation mapping

Explanatory only: normative text, SFR allocations, and Evaluation Activities control.

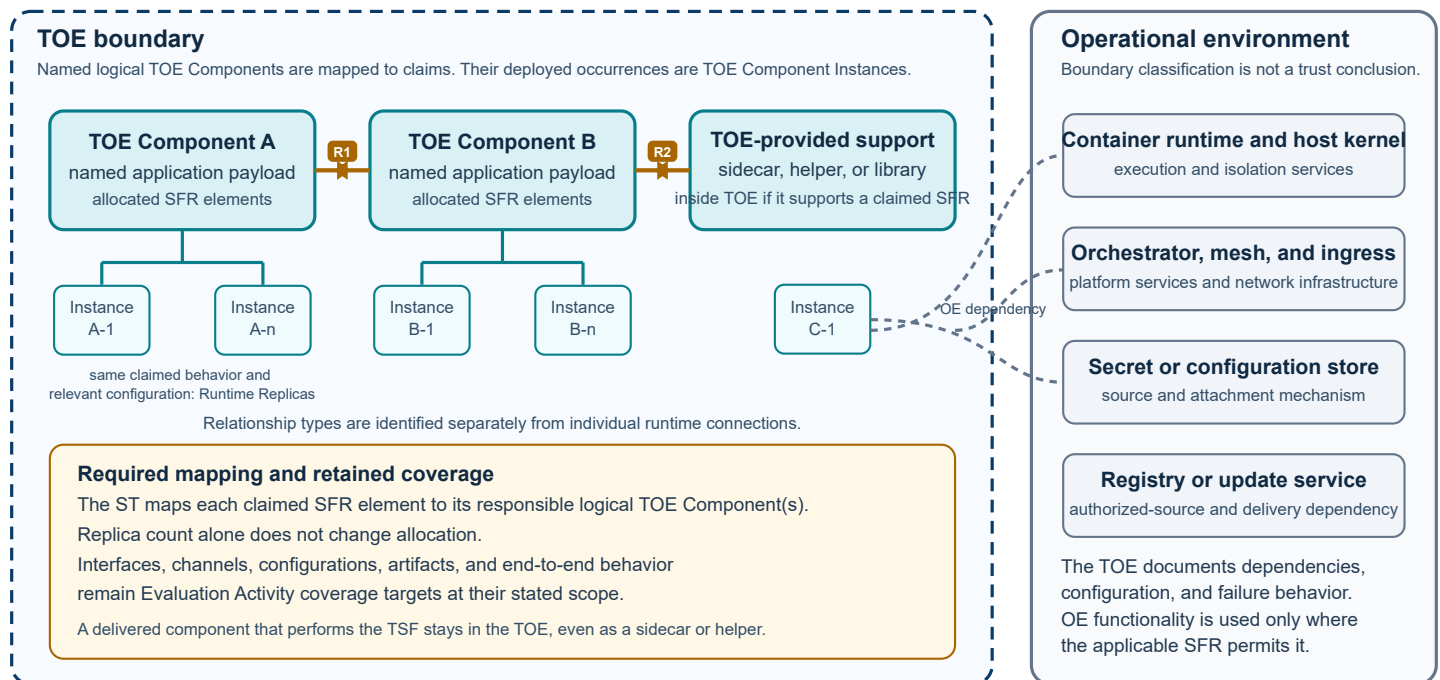


Figure 3: Illustrative Distributed TOE Boundary, Components, and Operational-Environment Dependencies

For containerized or microservices TOEs, the TOE consists of the application payloads and TOE-provided application components identified in the ST. Packaging does not determine the TOE boundary. A vendor-provided sidecar, init container, operator, library, userland utility, or other executable content that implements or supports a claimed SFR shall not be classified as operational environment merely because it is delivered or deployed using a container pattern. The container orchestration platform, container runtime, operating system kernel, platform service mesh infrastructure, ingress infrastructure, cluster networking, and platform-provided secret or configuration stores are part of the operational environment unless explicitly included in the TOE boundary. The PP does not require these operational environment components to be included in the TOE boundary solely because the TOE depends on them for execution, scheduling, networking, isolation, credential storage, configuration storage, or time services.

When the TOE relies on operational environment components to provide services used by the TOE, the ST shall identify the dependency and the guidance shall describe the required environmental configuration, expected service and failure behavior, and trust assumptions. An operational environment dependency may satisfy a claimed SFR only where that SFR expressly permits use of platform-provided functionality; otherwise the evaluator assesses the TOE's use of and dependency on the service, but the service does not satisfy a TOE-implemented requirement. A Communication Relationship Type is defined between TOE Components, which may be the same TOE Component, and realized by their TOE Component Instances. Communication between TOE Component Instances shall be identified in the ST. Where the TOE claims conformance to a PP-Configuration that includes Server and Agent application modules, the ST shall use the module requirements to address enablement, registration, and disablement of permitted communication pairings among TOE Component Instances and FTP_DIT_EXT.1 to address protection of data transmitted between those instances. For each claimed requirement or element, the ST shall provide an SFR allocation rationale that separately identifies the claim condition, where relevant; the component allocation; and any operational-environment contribution expressly allowed by the requirement. Component allocation shall be All Components, At Least One Component, or Feature Dependent. A Feature Dependent SFR or element is fulfilled only where the relevant feature is implemented by a distributed TOE Component. If it is included in the ST, at least one TOE Component shall implement that feature, and the ST shall map every implementing TOE Component to the SFR element or elements it implements. Where different TOE Components implement different elements of a multi-element SFR, each TOE Component shall satisfy only its mapped elements; the mapped elements collectively satisfy the TOE-level SFR. A feature, selection, or objective condition may control inclusion, but shall not omit an implementing TOE Component from the mapping or required coverage. The TOE as a whole shall satisfy every claimed SFR. Every TOE Component Instance shall exhibit the claimed SFR behavior allocated to its TOE Component in its identified configuration.

The ST need not iterate an SFR solely because the same completed requirement applies to more than one TOE Component. A single completion of the SFR, including one iteration when needed, and a common TSS description may be mapped to multiple TOE Components when the operations and claimed security-relevant behavior are the same. The ST shall identify every mapped TOE Component and shall use separate iterations when different operations are needed to express the claims. Multiple deployed instances of the same TOE Component are Runtime Replicas when they meet the Runtime Replica definition and may be represented by that TOE Component together with its permitted instance or scaling topology. Interfaces and Communication Relationship Types shall be identified separately as required by the applicable SFRs and Evaluation Activities.

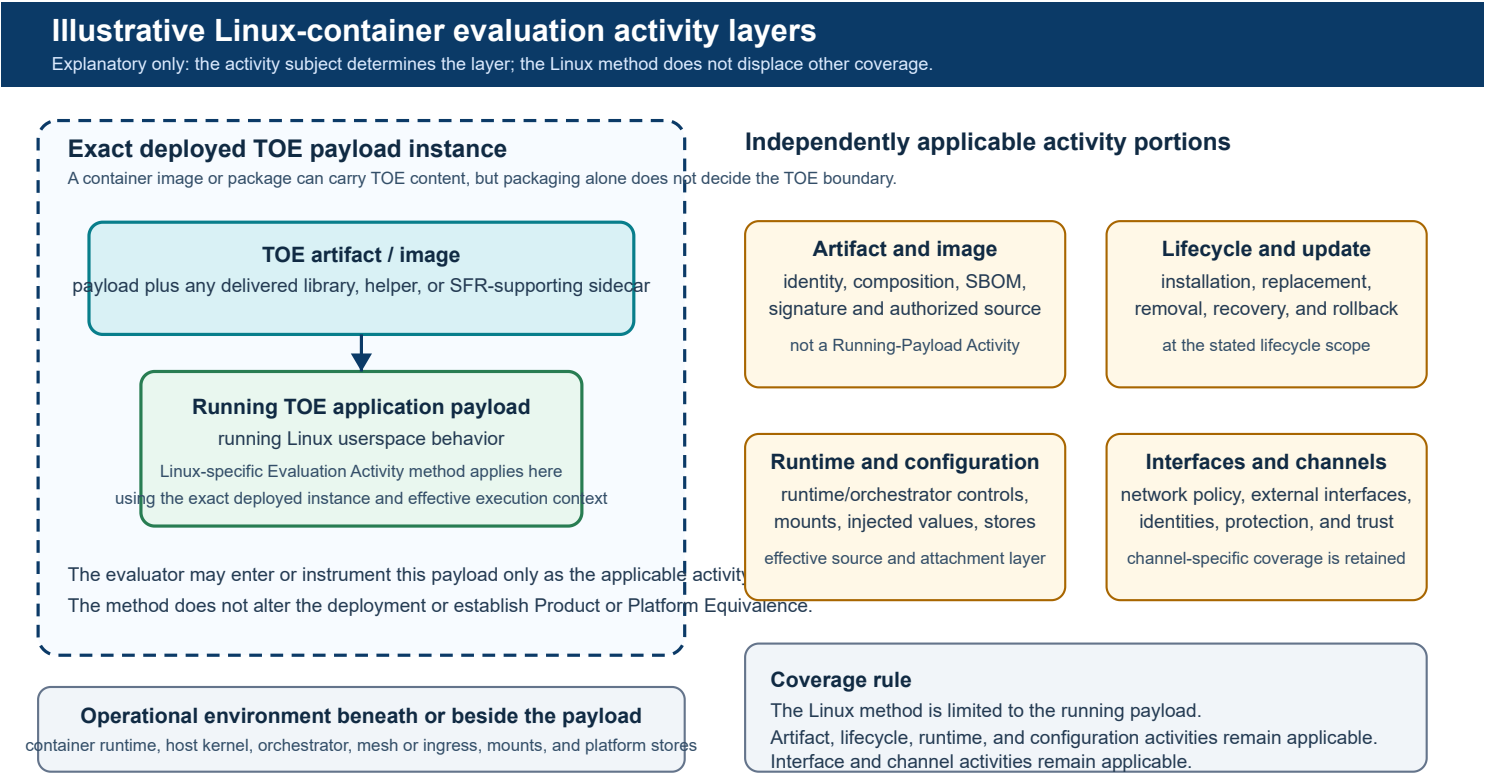
Where evidence reuse is claimed, the **ST** shall provide a concise mapping from each **TOE** Component and identified security-relevant configuration, where applicable, to its **SFR** iteration or iterations and implementation-equivalence class identifier. A single Component Implementation-Equivalence Class definition may list multiple **SFR** iterations when its member **SFR** Implementation Instances and equivalence basis are identical for those **SFR**s. The detailed rationale and the mapping from a class to the exact implementation-specific portions of Evaluation Activities for which representative execution is proposed may be provided in separate evaluation evidence and the evaluator's test plan. Class membership does not change **SFR** applicability, transfer compliance from one **TOE** Component or identified configuration to another, remove a **TOE** Component from the **TOE** boundary, or reduce interface, channel, configuration, or end-to-end coverage required by an Evaluation Activity. Component implementation equivalence and the associated evidence-reuse rules are described in [Appendix F - Component Implementation Equivalence Within a Distributed TOE](#).

1.6 Platforms with Specific EAs

This **PP** includes platform-specific EAs for the below-listed operating system platforms. For "bare-metal" applications, applications that run on other **OS** platforms, and any aspect of a software-based execution environment not covered by the limited Linux-container Running-Payload mapping below, contact the Technical Community for guidance.

- **Android:** *Mobile operating systems based on Google Android*
- **Microsoft Windows:** *Microsoft Windows operating systems*
- **Apple iOS and iPadOS:** *Apple's mobile operating system for iPhones and iPads*
- **Linux:** *Linux-based operating systems other than Android*
- **Oracle Solaris:** *Oracle's enterprise operating system*
- **Apple macOS:** *Apple's operating system for Mac devices*

For Evaluation Activity applicability, a **TOE** application payload executing in a Linux container shall be treated as the Linux platform case for each platform-specific Evaluation Activity whose subject is the running Linux userspace behavior of that payload, and the evaluator shall apply the Linux method to that Running-Payload Activity. This limited mapping does not establish Product Equivalence or Platform Equivalence and does not make any other activity inapplicable. Activity portions whose subject is image or package identity and authorized-source signature, image composition, installation, update or removal workflow, runtime or orchestrator configuration, mounts, networking, or an external interface or channel remain applicable at the artifact, platform, lifecycle, interface, or channel layer stated by the activity. The Supporting Document provides permitted methods and coverage rules for Running-Payload Activities.



2 Conformance Claims

Conformance Statement

An ST must claim exact conformance to this PP.

The evaluation methods used for evaluating the TOE are a combination of the workunits defined in [\[CEM\]](#) as well as the Evaluation Activities for ensuring that individual SFRs and SARs have a sufficient level of supporting evidence in the Security Target and guidance documentation and have been sufficiently tested by the laboratory as part of completing [ATE_IND.1](#). Any functional packages this PP claims similarly contain their own Evaluation Activities that are used in this same manner.

CC Conformance Claims

This PP is conformant to Part 2 (extended) and Part 3 (extended) of Common Criteria CC:2022, Revision 1.

PP Claim

This PP does not claim conformance to any Protection Profile.

The following PPs and PP-Modules are allowed to be specified in a PP-Configuration with this PP:

- Protection Profile for Mobile Device Management, Version 5.0
- PP-Module for Email Clients, Version 1.0
- PP-Module for Endpoint Detection and Response (EDR), Version 2.0
- PP-Module for File Encryption, Version 2.0
- PP-Module for File Encryption Enterprise Management, Version 2.0
- PP-Module for Application Software Agent, Version 2.0
- PP-Module for Application Software Server, Version 2.0
- PP-Module for Host Agent, Version 2.0
- PP-Module for Redaction Tools, Version 1.0
- PP-Module for Voice and Video over IP (VVoIP), Version 2.0
- PP-Module for VPN Client, Version 3.0
- PP-Module for Web Browsers, Version 1.0

Package Claim

- This PP is Functional Package for Secure Shell Version 2.0 conformant.
- This PP is Functional Package for Transport Layer Security Version 2.1 conformant.
- This PP is Functional Package for X.509 Version 1.0 conformant.
- This PP does not conform to any assurance packages.

The functional packages to which the PP conforms may include SFRs that are not mandatory to claim for the sake of conformance. An ST that claims one or more of these functional packages may include any non-mandatory SFRs that are appropriate to claim based on the capabilities of the TSE and on any triggers for their inclusion based inherently on the SFR selections made.

3 Security Problem Definition

The security problem is described in terms of the threats that the T.O.E is expected to address, assumptions about the operational environment, and any organizational security policies that the T.O.E is expected to enforce.

3.1 Threats

T.LOCAL_ATTACK

An attacker can act through unprivileged software on the same computing platform on which the application executes. Attackers may provide maliciously formatted input to the application in the form of files or other local communications.

T.NETWORK_ATTACK

An attacker is positioned on a communications channel or elsewhere on the network infrastructure. Attackers may engage in communications with the application software or alter communications between the application software and other endpoints in order to compromise it.

T.NETWORK_EAVESDROP

An attacker is positioned on a communications channel or elsewhere on the network infrastructure. Attackers may monitor and gain access to data exchanged between the application and other endpoints.

T.PHYSICAL_ACCESS

An attacker may try to access sensitive data at rest.

3.2 Assumptions

A.PLATFORM

The T.O.E relies upon a trustworthy computing platform with a reliable time clock for its execution. This includes the underlying platform and whatever runtime environment it provides to the T.O.E.

A.PROPER_ADMIN

The administrator of the application software is not careless, willfully negligent or hostile, and administers the software in compliance with the applied enterprise security policy.

A.PROPER_USER

The user of the application software is not willfully negligent or hostile, and uses the software in compliance with the applied enterprise security policy.

3.3 Organizational Security Policies

This P.P defines no Organizational Security Policies.

4 Security Objectives

4.1 Security Objectives for the Operational Environment

The following security objectives for the operational environment assist the `TOE` in correctly providing its security functionality. These track with the assumptions about the environment.

OE.PLATFORM

The `TOE` relies upon a trustworthy computing platform for its execution. This includes the underlying operating system and any discrete execution environment provided to the `TOE`.

OE.PROPER_ADMIN

The administrator of the application software is not careless, willfully negligent or hostile, and administers the software within compliance of the applied enterprise security policy.

OE.PROPER_USER

The user of the application software is not willfully negligent or hostile, and uses the software within compliance of the applied enterprise security policy.

4.2 Security Objectives Rationale

This section describes how the assumptions and organizational security policies map to operational environment security objectives.

Table 1: Security Objectives Rationale

Assumption or OSP	Security Objectives	Rationale
A.PLATFORM	OE.PLATFORM	The operational environment objective <code>OE.PLATFORM</code> is realized through <code>A.PLATFORM</code> .
A.PROPER_ADMIN	OE.PROPER_ADMIN	The operational environment objective <code>OE.PROPER_ADMIN</code> is realized through <code>A.PROPER_ADMIN</code> .
A.PROPER_USER	OE.PROPER_USER	The operational environment objective <code>OE.PROPER_USER</code> is realized through <code>A.PROPER_USER</code> .

5 Security Requirements

This chapter describes the security requirements which have to be fulfilled by the product under evaluation. Those requirements comprise functional components from Part 2 and assurance components from Part 3 of [CC]. The following conventions are used for the completion of operations:

- **Refinement** operation (denoted by **bold text** or ~~striktthrough text~~): Is used to add details to a requirement or to remove part of the requirement that is made irrelevant through the completion of another operation, and thus further restricts a requirement.
- **Selection** (denoted by *italicized text*): Is used to select one or more options provided by the [CC] in stating a requirement.
- **Assignment** operation (denoted by *italicized text*): Is used to assign a specific value to an unspecified parameter, such as the length of a password. Showing the value in square brackets indicates assignment.
- **Iteration** operation: Is indicated by appending the ~~SER~~ name with a slash and unique identifier suggesting the purpose of the operation, e.g. "/EXAMPLE1."

5.1 Security Functional Requirements

5.1.1 Cryptographic Support (FCS)

FCS_CKM_EXT.1 Cryptographic Key Generation Services

FCS_CKM_EXT.1.1

The application shall [**selection**:

- *generate no asymmetric cryptographic keys*
- *invoke platform-provided functionality for asymmetric key generation*
- *implement asymmetric key generation*

].

Application Note: If "**implement asymmetric key generation**" or "**invoke platform-provided functionality for asymmetric key generation**" is selected, then [FCS_CKM.1/AK](#) must be claimed in the ~~ST~~.

FCS_RBG_EXT.1 Random Bit Generation Services

FCS_RBG_EXT.1.1

The application shall [**selection**:

- *use no ~~DRBG~~ functionality*
- *invoke platform-provided ~~DRBG~~ functionality*
- *implement ~~DRBG~~ functionality*

] for its cryptographic operations.

Application Note: The selection "**invoke platform-provided ~~DRBG~~ functionality**" should only be chosen for direct invocations of the platform ~~DRBG~~ by the ~~TSE~~.

The selection "**use no ~~DRBG~~ functionality**" is chosen when the ~~TSE~~ calls a platform implementation of a function that subsequently calls a platform-provided ~~DRBG~~ itself, because this is not a direct invocation of the platform ~~DRBG~~ by the ~~TSE~~.

If "**implement ~~DRBG~~ functionality**" is selected, [FCS_RBG.1](#) must be claimed for the ~~DRBG~~ mechanism, and [FPT_TST.1](#) and [FPT_FLS.1](#) must be claimed for the self-testing and error handling of this mechanism.

In this requirement, cryptographic operations include all cryptographic key generation/derivation/agreement, IVs (for certain modes), as well as protocol-specific random values. Cryptographic operations in this requirement refer to the other cryptographic requirements in this ~~PP~~, not additional functionality that is not in scope.

FCS_STO_EXT.1 Storage of Credentials

FCS_STO_EXT.1.1

The application shall [**selection**:

- *not store any credentials*
 - *invoke the functionality provided by the platform to securely store [assignment: list of credentials]*
 - *securely store [assignment: list of credentials] with platform provided [selection:*
 - *[selection:*
 - *AES-CBC (as defined in NIST.SP.800-38A) mode*
 - *AES-GCM (as defined in NIST.SP.800-38D) mode*
 - *AES-XTS (as defined in NIST.SP.800-38E) mode*
 -] and cryptographic key size of 256-bits.*
 - *PBKDF2 function that uses [selection:*
 - *HMAC-SHA256*
 - *HMAC-SHA384*
 - *HMAC-SHA512*
 -] with [assignment: positive integer of 1,000 or greater] iterations and output cryptographic key size of [assignment: positive integer of 256 or greater] bits that meet the following [NIST.SP.800-132].*
 -]*
 - *implement functionality to securely store [assignment: list of credentials] according to [selection: FCS_COP.1/SKC, FCS_PBKDF_EXT.1]*
-] to non-volatile memory.*

Application Note: This requirement ensures that persistent credentials (secret keys, PKI private keys, passwords, etc) are stored securely, and never persisted in cleartext form. Application developers are encouraged to use platform mechanisms for the secure storage of credentials. Depending on the platform that may include hardware-backed protection for credential storage. Application developers must choose a selection, or multiple selections, based on all credentials that the application stores. If "not store any credentials" is selected, then the application must not store any credentials. If "invoke the functionality provided by the platform to securely store" is selected, then the application developer must closely review the EA for their platform and provide documentation indicating which platform mechanisms are used to store credentials. If "securely store" is selected, the application shall leverage platform cryptographic APIs to implement storage of credentials. If "implement functionality to securely store credentials" is selected, then the following components must be included in the ST: (FCS_COP.1/SKC and FCS_SNI_EXT.1) or FCS_PBKDF_EXT.1. If the QS is Linux and Java KeyStores are used to store credentials, "implement functionality to securely store credentials" must be selected.

5.1.2 User Data Protection (FDP)

FDP_DAR_EXT.1 Encryption Of Sensitive Application Data

FDP_DAR_EXT.1.1

The application shall [selection:

- *leverage platform-provided functionality to encrypt sensitive data*
- *implement functionality to encrypt sensitive data as defined in the PP-Module for File Encryption*
- *protect sensitive data in accordance with FCS_STO_EXT.1*
- *not store any sensitive data*

] in non-volatile memory.

Application Note: If "implement functionality to encrypt sensitive data as defined in the PP-Module for File Encryption" is selected, the TSF must claim conformance to a PP-Configuration that includes the PP-Module for File Encryption.

Any file that may potentially contain sensitive data (to include temporary files) shall be protected. The only exception is if the user intentionally exports the sensitive data to non-protected files. ST authors should select "protect sensitive data in accordance with FCS_STO_EXT.1" for the sensitive data that is covered by the FCS_STO_EXT.1 SFR.

FDP_DEC_EXT.1 Access to Platform Resources

FDP_DEC_EXT.1.1

The application shall restrict its access to only [selection:

- *no hardware resources*

- *network connectivity*
- *camera*
- *microphone*
- *location services*
- *NFC*
- *USB*
- *Bluetooth*
- *[assignment: list of additional hardware resources]*

].

Application Note: The intent is for the evaluator to ensure that the selection captures all hardware resources which the application accesses, and that these are restricted to those which are justified. On some platforms, the application must explicitly solicit permission in order to access hardware resources. Seeking such permissions, even if the application does not later make use of the hardware resource, should still be considered access. Selections should be expressed in a manner consistent with how the application expresses its access needs to the underlying platform. For example, the platform may provide location services which implies the potential use of a variety of hardware resources (e.g. satellite receivers, WiFi, cellular radio) yet "*location services*" is the proper selection. This is because use of these resources can be inferred, but also because the actual usage may vary based on the particular platform. Resources that do not need to be explicitly identified are those which are ordinarily used by any application such as central processing units, main memory, displays, input devices (e.g. keyboards, mice), and persistent storage devices provided by the platform.

FDP_DEC_EXT.1.2

The application shall restrict its access to only **[selection:**

- *no sensitive information repositories*
- *address book*
- *calendar*
- *call lists*
- *system logs*
- *[assignment: list of additional sensitive information repositories]*

].

Application Note: "Sensitive information repositories" are defined as those collections of sensitive data that could be expected to be shared among some applications, users, or user roles, but to which not all of these would ordinarily require access.

FDP_NET_EXT.1 Network Communications

FDP_NET_EXT.1.1

The application shall restrict network communication to **[selection:**

- *no network communication*
- *user-initiated communication for [assignment: list of functions for which the user can initiate network communication]*
- *respond to [assignment: list of remotely initiated communication]*
- *[assignment: list of application-initiated network communication]*

].

Application Note: This requirement is intended to restrict both inbound and outbound network communications to only those required, or to network communications that are user initiated. It does not apply to network communications in which the application may generically access the file system which may result in the platform accessing remotely mounted drives/shares.

5.1.3 Security Management (FMT)

FMT_CFG_EXT.1 Secure by Default Configuration

FMT_CFG_EXT.1.1

The application shall **[selection: not use credentials, use platform-provided credentials, provide only enough functionality to set new credentials when configured with default credentials or no credentials for application provided credentials]**.

Application Note: Default credentials are credentials (e.g., passwords, keys) that are automatically (without user interaction) loaded onto the platform during application installation. Credentials that are generated during installation using requirements laid out in [FCS_RBG_EXT.1](#) or established by

leveraging platform accounts are not by definition default credentials. For managed runtime or application framework based TOEs, default credentials include credentials, tokens, keys, or shared secrets exposed through runtime or framework defaults, sample configurations, development profiles, embedded management interfaces, or default administrative endpoints.

FMT_CFG_EXT.1.2

The application shall be configured by default with file permissions which protect the application binaries and data files from modification by normal unprivileged users.

Application Note: The precise expectations for file permissions vary per platform but the general intention is that a trust boundary protects the application and its data. For managed runtime or application framework based TOEs, this includes runtime or framework files bundled with the TOE, application packages, dependency archives, configuration files, plug-ins, classpath or module path locations, and other locations from which executable application or framework code may be loaded.

FMT_MEC_EXT.1 Supported Configuration Mechanism

FMT_MEC_EXT.1.1

The application shall [**selection:** *invoke the mechanisms recommended by the platform vendor for storing and setting configuration options, implement functionality to encrypt and store configuration options as defined by FDP_PRT_EXT.1 in the PP-Module for File Encryption*].

Application Note: Configuration options that are stored remotely are not subject to this requirement. Sensitive Data is generally not considered part of configuration options and should be stored according to FDP_DAR_EXT.1 or FCS_STO_EXT.1.

If “**implement functionality to encrypt and store configuration options as defined by FDP_PRT_EXT.1 in the PP-Module for File Encryption**” is selected, the TSE must claim conformance to a PP-Configuration that includes the PP-Module for File Encryption.

For managed runtime or application framework based TOEs, configuration options include security-relevant runtime or framework settings such as production or development profile selection, debug interfaces, remote management interfaces, management endpoints, routing, authentication and authorization settings, session management settings, object binding, serialization or deserialization settings, cryptographic provider selection, trust store configuration, plug-in loading, and external configuration source precedence.

FMT_SMF.1 Specification of Management Functions

FMT_SMF.1.1

The TSE shall be capable of performing the following management functions [**selection:**

- *no management functions*
- *enable/disable the transmission of any information describing the system's hardware, software, or configuration*
- *enable/disable the transmission of any PII*
- *enable/disable transmission of any application state (e.g. crashdump) information*
- *enable/disable network backup functionality to [**assignment:** list of enterprise or commercial cloud backup systems]*
- [**assignment:** list of other management functions to be provided by the TSE]

].

Application Note: This requirement stipulates that an application needs to provide the ability to enable/disable only those functions that it actually implements. The application is not responsible for controlling the behavior of the platform or other applications.

5.1.4 Privacy (FPR)

FPR_ANO_EXT.1 User Consent for Transmission of Personally Identifiable Information

FPR_ANO_EXT.1.1

The application shall [**selection, choose one of:**

- *not use PII*
- *not transmit PII over a network*
- *require user approval before executing [**assignment:** list of functions that transmit PII over a network]*

].

Application Note: PII is considered to be sensitive data. If "require user approval before executing..." is claimed, the ST must not claim "not transmit any..." in FTP_DIT_EXT.1.

This requirement applies only to PII that is specifically requested by the application; it does not apply if the user volunteers PII without prompting from the application into a general (or inappropriate) data field. A dialog box that declares intent to send PII presented to the user at the time the application is started is sufficient to meet this requirement.

5.1.5 Protection of the TSF (FPT)

FPT_AEX_EXT.1 Anti-Exploitation Capabilities

FPT_AEX_EXT.1.1

The application shall not request to map memory at an explicit address except for [assignment: list of explicit exceptions].

Application Note: Requesting a memory mapping at an explicit address subverts address space layout randomization (ASLR).

FPT_AEX_EXT.1.2

The application shall [selection, choose one of:

- not allocate any memory region with both write and execute permissions
- allocate memory regions with write and execute permissions for only [assignment: list of functions performing just-in-time compilation]

].

Application Note: Requesting a memory mapping with both write and execute permissions subverts the platform protection provided by DEP. If the application performs no just-in-time compiling, then the first selection must be chosen. If a managed runtime or framework that performs just-in-time compilation is included in the TOE boundary, the ST shall identify the runtime or framework functionality that requires write and execute memory and distinguish it from TOE application logic that does not require such memory permissions.

FPT_AEX_EXT.1.3

The application shall be compatible with security features provided by the platform vendor.

Application Note: This requirement is designed to ensure that platform security features do not need to be disabled in order for the application to run.

FPT_AEX_EXT.1.4

The application shall not write user-modifiable files to directories that contain executable files unless explicitly directed by the user to do so.

Application Note: The purpose of this requirement is to help ensure the integrity of application binaries by supporting file protection mechanisms such as directory-level file permissions and application allowlisting.

A user-modifiable file for purposes of this requirement is a file that is writable by an unprivileged user of the application -- either directly through application execution or independently of the application. If the application runs in the context of the application user, then the application should not be able to write to the directory containing the application binaries -- regardless of whether the files are configuration data, audit data, or temporary files.

Executables and user-modifiable files may not share the same parent directory, but may share directories above the parent.

FPT_AEX_EXT.1.5

The application shall be built with stack-based buffer overflow protection enabled.

FPT_API_EXT.1 Use of Supported Services and APIs

FPT_API_EXT.1.1

The application shall use only documented platform APIs.

Application Note: The definition of "documented" may vary depending upon whether the application is provided by a third party (who relies upon documented platform APIs) or by a platform vendor who may be able to guarantee support for platform APIs. For managed runtime or application framework based TOEs, documented APIs include supported runtime or framework APIs that mediate security-

relevant behavior, including APIs for code loading, native interface access, serialization or deserialization, object binding, authentication, authorization, session management, management endpoints, cryptographic providers, trust stores, configuration loading, and environment variable processing.

FPT_LIB_EXT.1 Use of Third Party Libraries

FPT_LIB_EXT.1.1

The application shall be packaged with only [assignment: *list of third-party libraries*].

Application Note: The intention of this requirement is for the evaluator to discover and document whether the application includes unnecessary or unexpected third-party libraries. This includes adware libraries which could present a privacy threat, as well as ensuring documentation of such libraries in case vulnerabilities are later discovered. For managed runtime or application framework based TOEs, the assignment includes bundled runtimes, embedded application servers, application frameworks, framework modules, transitive dependencies, shaded or repackaged dependencies, plug-ins, native interface libraries, and other executable dependency artifacts packaged with the TOE.

It is acceptable to complete the Assignment with a reference to the vendor-provided SBOM instead of listing the libraries in the ST.

FPT_TUD_EXT.1 Support for Trusted Updates

FPT_TUD_EXT.1.1

The application shall [selection: *provide the ability, use platform-provided services*] to check for updates and patches to the application software.

Application Note: This requirement is about the ability to "check" for updates. The actual installation of any updates should be done by the platform. This requirement is intended to ensure that the application can check for updates provided by the vendor, as updates provided by another source may contain malicious code. If the TOE includes a managed runtime or application framework in its TOE boundary, the update mechanism described shall cover updates to the managed runtime or application framework.

FPT_TUD_EXT.1.2

The application shall [selection: *provide the ability, use platform-provided services*] to query the current version of the application software.

FPT_TUD_EXT.1.3

The application shall [selection:

- *perform trusted updates*
- *not download, modify, replace or update its own binary code*

].

Application Note: This requirement applies to the code of the application; it does not apply to mobile code technologies that are designed for download and execution by the application.

If "perform trusted updates" is selected then [FPT_TUD_EXT.2](#) must be included in the ST.

FPT_TUD_EXT.1.4

Application updates shall be digitally signed such that the application platform can cryptographically verify them prior to installation.

Application Note: The specifics of the verification of updates involves requirements on the platform (and not the application), so these are not fully specified here.

FPT_TUD_EXT.1.5

The application is distributed [selection: *with the platform OS, as an additional software package to the platform OS*].

Application Note: Application software that is distributed as part of the platform operating system is not required to be packaged for installation or uninstallation. If "as an additional software package to the platform OS" is selected, the requirements from [FPT_TUD_EXT.2](#) must be included in the ST.

5.1.6 Trusted Path/Channels (FTP)

FTP_DIT_EXT.1 Protection of Data in Transit

The application shall [**selection**:

- not transmit any [**selection, choose one of**: data, sensitive data]
- encrypt all transmitted [**selection, choose one of**: sensitive data, data] with [**selection**:
 - [HTTPS](#) as a client in accordance with [FCS_HTTPS_EXT.1](#) and [FCS_HTTPS_EXT.2](#)
 - [HTTPS](#) as a server in accordance with [FCS_HTTPS_EXT.1](#)
 - [HTTPS](#) as a server with support for mutual authentication in accordance with [FCS_HTTPS_EXT.1](#) and [FCS_HTTPS_EXT.2](#)
 - [TLS](#) as a server as defined in Functional Package for Transport Layer Security (TLS), version 2.1 and also supports functionality for [**selection**: mutual authentication, none]
 - [TLS](#) as a client as defined in Functional Package for Transport Layer Security (TLS), version 2.1
 - [DTLS](#) as a server as defined in Functional Package for Transport Layer Security (TLS), version 2.1 and also supports functionality for [**selection**: mutual authentication, none]
 - [DTLS](#) as a client as defined in Functional Package for Transport Layer Security (TLS), version 2.1
 - [SSH](#) as defined in the [Functional Package for Secure Shell \(SSH\), version 2.0](#)
 - IPsec as defined in the [VPN Client PP-Module, version 3.0](#)
-] for [**assignment**: function(s)] using certificates as defined in the [Functional Package for X.509](#)
- invoke platform-provided functionality to encrypt all transmitted sensitive data with [**selection**: [HTTPS](#), [TLS](#), [DTLS](#), [SSH](#), IPsec] for [**assignment**: function(s)] using certificates as defined in the [Functional Package for X.509](#)
- invoke platform-provided functionality to encrypt all transmitted data with [**selection**: [HTTPS](#), [TLS](#), [DTLS](#), [SSH](#), IPsec] for [**assignment**: function(s)] using certificates as defined in the [Functional Package for X.509](#)

] between itself and another [**selection**: trusted IT product, TOE Component Instance].

Application Note: Encryption is not required for applications transmitting data that is not sensitive.

Where TOE Component Instance is selected as the peer, the ST shall identify each in-scope network-mediated intra-TOE Communication Relationship Type, the communication path, the TOE Components involved, any TOE Component Instances needed to describe the realized path, the protocol or mechanism used, the data or sensitive data transmitted, and whether the protection is implemented by the TOE, invoked from the platform, or provided through another allowed PP-Configuration dependency. A connection limited to the platform loopback interface is not subject to this requirement in this version of the CPP and does not require a trusted-channel claim or a separate [FTP_DIT_EXT.1](#) test. This scope boundary does not assert that local users or processes are trusted.

If "not transmit any..." is selected, no other option can be selected.

If "not transmit any..." is NOT selected, it is possible to select more than one of the other options to encrypt data for a specific cryptographic function (e.g., application encrypts management data using SSH AND application invokes platform-provided functionality to encrypt syslog data using TLS OR application encrypts syslog data using TLS. Protocol selections and function assignments should be made to cover all data/sensitive data.

If "encrypt all transmitted..." is selected and "TLS" or "DTLS" as a client or server is selected, then corresponding components from Functional Package for Transport Layer Security (TLS), version 2.1 must be selected.

If "encrypt all transmitted..." is selected and any claim involving [HTTPS](#) is selected, then [FCS_HTTPS_EXT.1](#) and potentially [FCS_HTTPS_EXT.2](#) is required, as indicated by the chosen selections.

If "encrypt all transmitted..." is selected and "SSH" is selected, then the TSF shall be validated against the [Functional Package for Secure Shell](#).

If "encrypt all transmitted..." is selected and "IPsec" is selected, then the TSF must claim conformance to a [PP-Configuration that includes the VPN Client PP-Module, version 3.0](#).

If "encrypt all transmitted..." is selected, [FCS_CKM.2](#) and all iterations of [FCS_COP.1](#) must be claimed.

Claims from the [Functional Package for X.509](#) are only required to the extent that they are needed to support the functionality required by the trusted protocols that are claimed. For example, if the TOE supports [HTTPS](#) as a server but does not support mutual authentication, then for this interface the TSF would only present certificates in accordance with the requirements of the package and not validate presented certificates.

If the **T.SF** implements a protocol that requires the validation of a certificate presented by an external entity, **FIA_X509_EXT.1** and **FIA_X509_EXT.2** will be claimed. **FIA_TSM_EXT.1** may also be claimed if the **T.SF** implements its own trust store. Note that **FIA_X509_EXT.1** and **FIA_X509_EXT.2** have selections for invocation of platform-provided functionality, so it is expected that these claims are made and tested even when the trusted protocol is implemented by the **T.OE** platform.

If the **T.SF** implements a protocol that requires the presentation of any certificates to an external entity, **FIA_XCU_EXT.2** from [Functional Package for X.509](#) will be claimed. **FIA_X509_EXT.3** from [Functional Package for X.509](#) will also be claimed, along with any applicable dependencies, depending on how the certificates presented by the **T.OE** are obtained.

If the **T.SF** implements a protocol that does not require presenting or validating X.509 certificates, no claims from the [Functional Package for X.509](#) are required.

5.1.7 TOE Security Functional Requirements Rationale

The following rationale provides justification for each **SFR** for the **T.OE**, showing that the **SFRs** are suitable to address the specified threats:

Table 2: SFR Rationale

Threat	Addressed by	Rationale
T.LOCAL_ATTACK	FCS_CKM_EXT.1	The PP includes FCS_CKM_EXT.1 to specify that the T.SF may rely on platform-provided key generation services.
	FCS_RBG_EXT.1	The PP includes FCS_RBG_EXT.1 to specify that the T.SF may rely on platform-provided random bit generation services.
	FCS_STO_EXT.1	The PP includes FCS_STO_EXT.1 to specify that the T.SF may rely on platform-provided credential storage services.
	FDP_DAR_EXT.1	The PP includes FDP_DAR_EXT.1 to specify that the T.SF may rely on platform-provided data-at-rest protection services.
	FDP_DEC_EXT.1	The PP includes FDP_DEC_EXT.1 to limit access to platform hardware resources, which limits the methods by which an attacker can attempt to locally compromise the integrity of the T.OE .
	FMT_CFG_EXT.1	The PP includes FMT_CFG_EXT.1 for the TSP to limit unauthorized access to itself by preventing the use of default authentication credentials and by ensuring that the T.OE uses appropriately restrictive platform permissions on its binaries and data
	FMT_MEC_EXT.1	The PP includes FMT_MEC_EXT.1 to ensure that the T.OE can use platform services to store and set configuration options.
	FPT_AEX_EXT.1	The PP includes FPT_AEX_EXT.1 to add complexity to the task of compromising systems by ensuring that the T.OE implements various platform security features and can operate on a platform that is configured securely.
	FPT_API_EXT.1	The PP includes FPT_API_EXT.1 to require the T.OE to leverage platform functionality by using only documented and supported APIs.
	FPT_LIB_EXT.1	The PP includes FPT_LIB_EXT.1 to ensure that the T.OE does not include any unnecessary or unexpected third-party libraries which could present a privacy threat or vulnerability.
	FPT_TUD_EXT.1	The PP includes FPT_TUD_EXT.1 to ensure that the T.OE can be patched and that any updates to the T.OE have appropriate integrity protection.
	FPT_IPC_EXT.1 (Objective)	The PP includes FPT_IPC_EXT.1 to provide an objective for protection of non-network local inter-process communication between T.OE Component Instances.
	FPT_API_EXT.2 (Objective)	The PP includes FPT_API_EXT.2 to permit the T.OE to use platform-provided libraries for parsing IANA MIME media formats.
	FCS_CKM.1/AK (Selection-based)	The PP includes FCS_CKM.1/AK to specify that the T.SF may rely on platform-provided asymmetric key generation services.

	FCS_CKM.2 (Selection-based)	The PP includes FCS_CKM.2 to specify that the TSE may rely on platform-provided key establishment services.
	FPT_TUD_EXT.2 (Selection-based)	The PP includes FPT_TUD_EXT.2 to ensure that TOE updates are packaged in a certain format, provide certain integrity protections, and remove residual data.
T.NETWORK_ATTACK	FCS_CKM_EXT.1	The PP includes FCS_CKM_EXT.1 to specify whether the TOE or the platform is responsible for generation of any asymmetric keys that may be used for establishing trusted communications.
	FCS_RBG_EXT.1	The PP includes FCS_RBG_EXT.1 to define whether the random bit generation services used in establishing trusted communications are implemented by the TSE or by the platform.
	FDP_DEC_EXT.1	The PP includes FDP_DEC_EXT.1 to limit access to platform hardware resources, which limits the methods by which an attacker can attempt to remotely compromise the integrity of the TOE.
	FDP_NET_EXT.1	The PP includes FDP_NET_EXT.1 to define the TOE's usage of network communications, which may include the transmission or receipt of data over a trusted channel.
	FMT_CFG_EXT.1	The PP includes FMT_CFG_EXT.1 for the TSP to limit unauthorized access to itself by preventing the use of default authentication credentials and by ensuring that the TOE uses appropriately restrictive platform permissions on its binaries and data
	FMT_SMF.1	The PP includes FMT_SMF.1 to define the security-relevant management functions that are supported by the TOE, which may include configuration of network behavior.
	FPR_ANO_EXT.1	The PP includes FPR_ANO_EXT.1 to define how the TSE provides control to the user regarding the disclosure of any PII.
	FPT_AEX_EXT.1	The PP includes FPT_AEX_EXT.1 to add complexity to the task of compromising systems by ensuring that the TOE implements various platform security features and can operate on a platform that is configured securely.
	FPT_TUD_EXT.1	The PP includes FPT_TUD_EXT.1 to ensure that updates to the TOE have integrity protection and cannot be altered via network attack.
	FTP_DIT_EXT.1	The PP includes FTP_DIT_EXT.1 to define the trusted channels used to protect data in transit, the data that is protected, and whether the trusted channels are implemented by the TSE or the platform.
	FPT_IDV_EXT.1 (Objective)	The PP includes FPT_IDV_EXT.1 to provide a mechanism to identify the TOE version so that it can be determined whether a vulnerability is present on the system based on the installed version.
	FCS_CKM.1/AK (Selection-based)	The PP includes FCS_CKM.1/AK to define whether the TSE or the platform generates asymmetric keys that are used in support of trusted communications.
	FCS_CKM.1/SK (Selection-based)	The PP includes FCS_CKM.1/SK to define the mechanism used to generate symmetric keys when the TOE performs this function.
	FCS_CKM.2 (Selection-based)	The PP includes FCS_CKM.2 to define whether the TSE or the platform performs key establishment for trusted communications.
	FCS_COP.1/Hash (Selection-based)	The PP includes FCS_COP.1/Hash to define the hash algorithms used in support of trusted communications.
	FCS_COP.1/KeyedHash (Selection-based)	The PP includes FCS_COP.1/KeyedHash to define the HMAC algorithms used in support of trusted communications.
	FCS_COP.1/SigGen (Selection-based)	The PP includes FCS_COP.1/SigGen to define the digital signature algorithms used in support of trusted communications.

	FCS_COP.1/SigVer (Selection-based)	The PP includes FCS_COP.1/SigVer to define the digital signature algorithms used in support of trusted communications and trusted updates.
	FCS_COP.1/SKC (Selection-based)	The PP includes FCS_COP.1/SKC to define the symmetric encryption algorithms used in support of trusted communications.
	FCS_HTTPS_EXT.1 (Selection-based)	The PP includes FCS_HTTPS_EXT.1 to define the TOE's support for the HTTPS trusted communications protocol.
	FCS_HTTPS_EXT.2 (Selection-based)	The PP includes FCS_HTTPS_EXT.2 to define the TOE's handling of X.509 certificates in the context of HTTPS communications.
	FCS_RBG.1 (Selection-based)	The PP includes FCS_RBG.1 to define the DRBG algorithms used in support of trusted communications.
	FCS_RBG.2 (Selection-based)	The PP includes FCS_RBG.2 to define how entropy is obtained for secure DRBG seeding.
	FCS_RBG.3 (Selection-based)	The PP includes FCS_RBG.3 to define how entropy is obtained for secure DRBG seeding.
	FCS_RBG.4 (Selection-based)	The PP includes FCS_RBG.4 to define how entropy is obtained for secure DRBG seeding.
	FCS_RBG.5 (Selection-based)	The PP includes FCS_RBG.5 to define how entropy is obtained for secure DRBG seeding.
	FCS_SNI_EXT.1 (Selection-based)	The PP includes FCS_SNI_EXT.1 to define the proper salt, nonce, and initialization vector usage to ensure proper cryptographic operation.
	FPT_FLS.1 (Selection-based)	The PP includes FPT_FLS.1 to ensure that the TSF will not operate when it is in a state where it is unable to generate secure random numbers.
	FPT_TST.1 (Selection-based)	The PP includes FPT_TST.1 to ensure that the TSF can determine whether or not it is capable of generating secure random numbers.
	FPT_TUD_EXT.2 (Selection-based)	The PP includes FPT_TUD_EXT.2 to define specific integrity protections for certain types of updates.
T.NETWORK_EAVESDROP	FCS_CKM_EXT.1	The PP includes FCS_CKM_EXT.1 to specify whether the TOE or the platform is responsible for generation of any asymmetric keys that may be used for establishing trusted communications.
	FCS_RBG_EXT.1	The PP includes FCS_RBG_EXT.1 to define whether the random bit generation services used in establishing trusted communications are implemented by the TSF or by the platform.
	FCS_STO_EXT.1	The PP includes FCS_STO_EXT.1 to specify that the TSF may rely on platform-provided credential storage services.
	FDP_DAR_EXT.1	The PP includes FDP_DAR_EXT.1 to specify that the TSF may rely on platform-provided data-at-rest protection services.
	FDP_NET_EXT.1	The PP includes FDP_NET_EXT.1 to define the TOE's usage of network communications, which may include the transmission or receipt of data over a trusted channel.
	FMT_MEC_EXT.1	The PP includes FMT_MEC_EXT.1 to ensure that the TOE can use platform services to store and set configuration options.
	FMT_SMF.1	The PP includes FMT_SMF.1 to define the security-relevant management functions that are supported by the TOE.
	FPR_ANO_EXT.1	The PP includes FPR_ANO_EXT.1 to define how the TSF provides control to the user regarding the disclosure of any PII.

FPT_API_EXT.1	The PP includes FPT_API_EXT.1 to require the TOE to leverage platform functionality by using only documented and supported APIs.
FPT_LIB_EXT.1	The PP includes FPT_LIB_EXT.1 to ensure that the TOE does not include any unnecessary or unexpected third-party libraries which could present a privacy threat or vulnerability.
FTP_DIT_EXT.1	The PP includes FTP_DIT_EXT.1 to define the trusted channels used to protect data in transit, the data that is protected, and whether the trusted channels are implemented by the TSF or the platform.
FPT_API_EXT.2 (Objective)	The PP includes FPT_API_EXT.2 to permit the TOE to use platform-provided libraries for parsing IANA MIME media formats.
FPT_IDV_EXT.1 (Objective)	The PP includes FPT_IDV_EXT.1 to provide a mechanism to identify the TOE version so that it can be determined whether a vulnerability is present on the system based on the installed version.
FCS_CKM.1/AK (Selection-based)	The PP includes FCS_CKM.1/AK to define whether the TSF or the platform generates asymmetric keys that are used in support of trusted communications.
FCS_CKM.1/SK (Selection-based)	The PP includes FCS_CKM.1/SK to define the mechanism used to generate symmetric keys when the TOE performs this function.
FCS_CKM.2 (Selection-based)	The PP includes FCS_CKM.2 to define whether the TSF or the platform performs key establishment for trusted communications.
FCS_COP.1/Hash (Selection-based)	The PP includes FCS_COP.1/Hash to define the hash algorithms used in support of trusted communications.
FCS_COP.1/KeyedHash (Selection-based)	The PP includes FCS_COP.1/KeyedHash to define the HMAC algorithms used in support of trusted communications.
FCS_COP.1/SigVer (Selection-based)	The PP includes FCS_COP.1/SigVer to define the mechanism used to verify TOE updates if the TOE implements this functionality rather than the underlying platform.
FCS_COP.1/SKC (Selection-based)	The PP includes FCS_COP.1/SKC to define the symmetric encryption algorithms used in support of trusted communications.
FCS_HTTPS_EXT.1 (Selection-based)	The PP includes FCS_HTTPS_EXT.1 to define the TOE's support for the HTTPS trusted communications protocol.
FCS_HTTPS_EXT.2 (Selection-based)	The PP includes FCS_HTTPS_EXT.2 to define the TOE's handling of X.509 certificates in the context of HTTPS communications.
FCS_RBG.1 (Selection-based)	The PP includes FCS_RBG.1 to define the DRBG algorithms used in support of trusted communications.
FCS_RBG.2 (Selection-based)	The PP includes FCS_RBG.2 to define how entropy is obtained for secure DRBG seeding.
FCS_RBG.3 (Selection-based)	The PP includes FCS_RBG.3 to define how entropy is obtained for secure DRBG seeding.
FCS_RBG.4 (Selection-based)	The PP includes FCS_RBG.4 to define how entropy is obtained for secure DRBG seeding.
FCS_RBG.5 (Selection-based)	The PP includes FCS_RBG.5 to define how entropy is obtained for secure DRBG seeding.
FPT_FLS.1 (Selection-based)	The PP includes FPT_FLS.1 to ensure that the TSF will not operate when it is in a state where it is unable to generate secure random numbers.
FPT_TST.1 (Selection-based)	The PP includes FPT_TST.1 to ensure that the TSF can determine whether or not it is capable of generating secure random numbers.

T.PHYSICAL_ACCESS	FCS_RBG_EXT.1	The PP includes FCS_RBG_EXT.1 to define whether random bit generation services are implemented by the TSF or the platform. Depending on how data at rest is protected, the TOE may rely on the use of a random bit generator to create keys that are subsequently used for data protection.
	FCS_STO_EXT.1	The PP includes FCS_STO_EXT.1 to define the mechanism that the TSF uses or relies upon to protect stored credential data.
	FDP_DAR_EXT.1	The PP includes FDP_DAR_EXT.1 to define the mechanism that the TSF uses or relies upon to protect sensitive data at rest.
	FPT_IDV_EXT.1 (Objective)	The PP includes FPT_IDV_EXT.1 to provide a mechanism to identify the TOE version so that it can be determined whether a vulnerability is present on the system based on the installed version.
	FCS_CKM.1/SK (Selection-based)	The PP includes FCS_CKM.1/SK to define the TOE's capability to generate symmetric keys. These keys may subsequently be used to encrypt stored credential data based on the claims made in FCS_STO_EXT.1.
	FCS_COP.1/Hash (Selection-based)	The PP includes FCS_COP.1/Hash to define integrity mechanisms that may be used by the TOE as part of ensuring that data at rest is protected.
	FCS_COP.1/KeyedHash (Selection-based)	The PP includes FCS_COP.1/KeyedHash to define HMAC mechanisms that may be used by the TOE as part of ensuring that data at rest is protected.
	FCS_COP.1/SKC (Selection-based)	The PP includes FCS_COP.1/SKC to define the AES cryptographic algorithm that may be used to encrypt stored credential data based on the claims made in FCS_STO_EXT.1.
	FCS_PBKDF_EXT.1 (Selection-based)	The PP includes FCS_PBKDF_EXT.1 to define the password-based key derivation function that may be used to encrypt stored credential data based on the claims made in FCS_STO_EXT.1.
	FCS_RBG.1 (Selection-based)	The PP includes FCS_RBG.1 to define the DRBG algorithms used in support of trusted communications.
	FCS_RBG.2 (Selection-based)	The PP includes FCS_RBG.2 to define how entropy is obtained for secure DRBG seeding.
	FCS_RBG.3 (Selection-based)	The PP includes FCS_RBG.3 to define how entropy is obtained for secure DRBG seeding.
	FCS_RBG.4 (Selection-based)	The PP includes FCS_RBG.4 to define how entropy is obtained for secure DRBG seeding.
	FCS_RBG.5 (Selection-based)	The PP includes FCS_RBG.5 to define how entropy is obtained for secure DRBG seeding.
	FPT_FLS.1 (Selection-based)	The PP includes FPT_FLS.1 to ensure that the TSF will not operate when it is in a state where it is unable to generate secure random numbers.
	FPT_TST.1 (Selection-based)	The PP includes FPT_TST.1 to ensure that the TSF can determine whether or not it is capable of generating secure random numbers.

5.2 Security Assurance Requirements

The PP identifies the Security Assurance Requirements (SARs) to frame the extent to which the evaluator assesses the documentation applicable for the evaluation and performs independent testing.

This section lists the set of SARs from CC part 3 that are required in evaluations against this PP. Individual Evaluation Activities (EAs) to be performed are specified both in [Section 5 Security Requirements](#) as well as in this section. These SARs were chosen based on the notion that a hypothetical attacker of the TOE lacks administrative privilege on its platform but otherwise has persistent access to the TOE itself and the sophistication to interact with the platform in a way that they can attempt to access stored data without authorization or to run tools that automate more sophisticated malicious activity.

The general model for evaluation of TOEs against STs written to conform to this PP is as follows:

After the ST has been approved for evaluation, the CCTL will obtain the TOE, supporting environmental IT, and the administrative/user guides for the TOE. The CCTL is expected to perform actions mandated by the Common Evaluation Methodology (CEM) for the ASE and ALC SARs. The CCTL also performs the evaluation activities contained within [Section 5 Security Requirements](#), which are intended to be an interpretation of the other CEM assurance requirements as they apply to the specific technology instantiated in the TOE. The evaluation activities that are captured in [Section 5 Security Requirements](#) also provide clarification as to what the developer needs to provide to demonstrate the TOE is compliant with the PP. The results of these activities will be documented and presented (along with the administrative guidance used) for validation.

5.2.1 Class ASE: Security Target

As per ASE activities defined in [\[CEM\]](#).

5.2.2 Class ADV: Development

The information about the TOE is contained in the guidance documentation available to the end user as well as the TSS portion of the ST. The TOE developer must concur with the description of the product that is contained in the TSS as it relates to the functional requirements. The evaluation activities contained in [Section 5.1 Security Functional Requirements](#) should provide the ST authors with sufficient information to determine the appropriate content for the TSS section.

ADV_FSP.1 Basic Functional Specification (ADV_FSP.1)

The functional specification describes the TSFIs. It is not necessary to have a formal or complete specification of these interfaces. Additionally, because TOEs conforming to this PP will necessarily have interfaces to the Operational Environment that are not directly invocable by TOE users, there is little point specifying that such interfaces be described in and of themselves since only indirect testing of such interfaces may be possible. For this PP, the activities for this family should focus on understanding the interfaces presented in the TSS in response to the functional requirements and the interfaces presented in the AGD documentation. No additional “functional specification” documentation is necessary to satisfy the evaluation activities specified. The interfaces that need to be evaluated are characterized through the information needed to perform the assurance activities listed, rather than as an independent, abstract list.

Developer action elements:

ADV_FSP.1.1D

The developer shall provide a functional specification.

ADV_FSP.1.2D

The developer shall provide a tracing from the functional specification to the SERs.

Application Note: As indicated in the introduction to this section, the functional specification is comprised of the information contained in the AGD_OPE and AGD_PRE documentation. The developer may reference a website accessible to application developers and the evaluator. The evaluation activities in the functional requirements point to evidence that should exist in the documentation and TSS section; since these are directly associated with the SERs, the tracing in element [ADV_FSP.1.2D](#) is implicitly already done and no additional documentation is necessary.

Content and presentation elements:

ADV_FSP.1.1C

The functional specification shall describe the purpose and method of use for each SER-enforcing and SER-supporting TSFI.

ADV_FSP.1.2C

The functional specification shall identify all parameters associated with each SER-enforcing and SER-supporting TSFI.

ADV_FSP.1.3C

The functional specification shall provide rationale for the implicit categorization of interfaces as SER-non-interfering.

ADV_FSP.1.4C

The tracing shall demonstrate that the SERs trace to TSFIs in the functional specification.

Evaluator action elements:

ADV_FSP.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

The evaluator shall determine that the functional specification is an accurate and complete instantiation of the SFRs.

5.2.3 Class AGD: Guidance Documentation

The guidance documents will be provided with the ST. Guidance must include a description of how the IT personnel verifies that the Operational Environment can fulfill its role for the security functionality. The documentation should be in an informal style and readable by the IT personnel. Guidance must be provided for every operational environment that the product supports as claimed in the ST. This guidance includes instructions to successfully install the TSF in that environment; and instructions to manage the security of the TSF as a product and as a component of the larger operational environment. Guidance pertaining to particular security functionality is also provided; requirements on such guidance are contained in the evaluation activities specified with each requirement.

AGD_OPE.1 Operational User Guidance (AGD_OPE.1)

Developer action elements:

AGD_OPE.1.1D

The developer shall provide operational user guidance.

Application Note: The operational user guidance does not have to be contained in a single document. Guidance to users, administrators and application developers can be spread among documents or web pages. Where appropriate, the guidance documentation is expressed in the eXtensible Configuration Checklist Description Format (XCCDF) to support security automation. Rather than repeat information here, the developer should review the evaluation activities for this component to ascertain the specifics of the guidance that the evaluator will be checking for. This will provide the necessary information for the preparation of acceptable guidance.

Content and presentation elements:

AGD_OPE.1.1C

The operational user guidance shall describe, for each user role, the user-accessible functions and privileges that should be controlled in a secure processing environment, including appropriate warnings.

Application Note: User and administrator are to be considered in the definition of user role.

AGD_OPE.1.2C

The operational user guidance shall describe, for each user role, how to use the available interfaces provided by the TOE in a secure manner.

AGD_OPE.1.3C

The operational user guidance shall describe, for each user role, the available functions and interfaces, in particular all security parameters under the control of the user, indicating secure values as appropriate.

AGD_OPE.1.4C

The operational user guidance shall, for each user role, clearly present each type of security-relevant event relative to the user-accessible functions that need to be performed, including changing the security characteristics of entities under the control of the TSF.

AGD_OPE.1.5C

The operational user guidance shall identify all possible modes of operation of the TOE (including operation following failure or operational error), their consequences, and implications for maintaining secure operation.

AGD_OPE.1.6C

The operational user guidance shall, for each user role, describe the security measures to be followed in order to fulfill the security objectives for the operational environment as described in the ST.

AGD_OPE.1.7C

The operational user guidance shall be clear and reasonable.

Evaluator action elements:

AGD_OPE.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AGD_PRE.1 Preparative Procedures (AGD_PRE.1)

Developer action elements:

AGD_PRE.1.1D

The developer shall provide the TOE, including its preparative procedures.

Application Note: As with the operational guidance, the developer should look to the evaluation activities to determine the required content with respect to preparative procedures.

Content and presentation elements:

AGD_PRE.1.1C

The preparative procedures shall describe all the steps necessary for secure acceptance of the delivered TOE in accordance with the developer's delivery procedures.

AGD_PRE.1.2C

The preparative procedures shall describe all the steps necessary for secure installation of the TOE and for the secure preparation of the operational environment in accordance with the security objectives for the operational environment as described in the ST.

Evaluator action elements:

AGD_PRE.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AGD_PRE.1.2E

The evaluator shall apply the preparative procedures to confirm that the TOE can be prepared securely for operation.

5.2.4 Class ALC: Life-cycle Support

At the assurance level provided for TOEs conformant to this PP, life-cycle support is limited to end-user-visible aspects of the life-cycle, rather than an examination of the TOE vendor's development and configuration management process. This is not meant to diminish the critical role that a developer's practices play in contributing to the overall trustworthiness of a product; rather, it is a reflection on the information to be made available for evaluation at this assurance level.

ALC_CMC.1 Labeling of the TOE (ALC_CMC.1)

This component is targeted at identifying the TOE such that it can be distinguished from other products or versions from the same vendor and can be easily specified when being procured by an end user.

Developer action elements:

ALC_CMC.1.1D

The developer shall provide the TOE and a reference for the TOE.

Content and presentation elements:

ALC_CMC.1.1C

The application shall be labeled with a unique reference.

Application Note: Unique reference information includes:

- Application Name
- Application Version
- Application Description
- Platform on which Application Runs
- Software Identification (SWID) tags, if available

Evaluator action elements:

ALC_CMC.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_CMS.1 TOE CM Coverage (ALC_CMS.1)

Developer action elements:

ALC_CMS.1.1D

The developer shall provide a configuration list for the TOE.

Content and presentation elements:

ALC_CMS.1.1C

The configuration list shall include the following: the TOE itself; and the evaluation evidence required by the SARs.

ALC_CMS.1.2C

The configuration list shall uniquely identify the configuration items.

Evaluator action elements:

ALC_CMS.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_TSU_EXT.1 Timely Security Updates

This component requires the TOE developer, in conjunction with any other necessary parties, to provide information as to how the end-user devices are updated to address security issues in a timely manner. The documentation describes the process of providing updates to the public from the time a security flaw is reported/discovered, to the time an update is released. This description includes the parties involved (e.g., the developer, carriers(s)) and the steps that are performed (e.g., developer testing, carrier testing), including worst case time periods, before an update is made available to the public.

Developer action elements:

ALC_TSU_EXT.1.1D

The developer shall provide a description in the TSS of how timely security updates are made to the TOE.

Application Note: Application developers must support updates to their products for purposes of fixing security vulnerabilities.

ALC_TSU_EXT.1.2D

The developer shall provide a description in the TSS of how users are notified when updates change security properties or the configuration of the product.

Content and presentation elements:

ALC_TSU_EXT.1.1C

The description shall include the process for creating and deploying security updates for the TOE software.

ALC_TSU_EXT.1.2C

The description shall express the time window as the length of time, in days, between public disclosure of a vulnerability and the public availability of security updates to the TOE.

ALC_TSU_EXT.1.3C

The description shall include the mechanisms publicly available for reporting security issues pertaining to the TOE.

Application Note: The reporting mechanism could include a website or email address as well as a means to protect the sensitive nature of the report (e.g., public keys that could be used to encrypt the details of a proof-of-concept exploit).

Evaluator action elements:

ALC_TSU_EXT.1.1E

The evaluator *shall confirm* that the information provided meets all requirements for content and presentation of evidence.

5.2.5 Class ATE: Tests

Testing is specified for functional aspects of the system as well as aspects that take advantage of design or implementation weaknesses. The former is done through the ATE_IND family, while the latter is through the AVA_VAN family. At the assurance level specified in this PP, testing is based on advertised functionality and interfaces with dependency on the availability of design information. One of the primary outputs of the evaluation process is the test report as specified in the following requirements.

ATE_IND.1 Independent Testing – Conformance (ATE_IND.1)

Testing is performed to confirm the functionality described in the TSS as well as the administrative (including configuration and operational) documentation provided. The focus of the testing is to confirm that the requirements specified in [Section 5.1 Security Functional Requirements](#) are being met, although some additional testing is specified for SARs in [Section 5.2 Security Assurance Requirements](#). The evaluation activities identify the additional testing activities associated with these components. The evaluator produces a test report documenting the plan for and results of testing, as well as coverage arguments focused on the platform/TOE combinations that are claiming conformance to this PP. Given the scope of the TOE and its associated evaluation evidence requirements, this component's evaluation activities are covered by the evaluation activities listed for [ALC_CMC.1](#).

Developer action elements:

ATE_IND.1.1D

The developer shall provide the TOE for testing.

Application Note: The developer must provide one or more complete deployed TOE configurations for testing on at least one platform regardless of equivalency. Collectively, the deployed configurations shall include at least one exact TOE Component Instance representing every claimed TOE Component, including every identified security-relevant configuration of each TOE Component where applicable, and every materially distinct Communication Relationship Type and Protected Channel Instance, unless an optional TOE Component, configuration, or topology that is not instantiated is identified, justified, and covered as required by the applicable activities. Additional Runtime Replicas need not all be instantiated solely because replica count can scale; failover, scaling, placement, or alternate-topology behavior shall be included when it creates a distinct security-relevant path or configuration. Representative execution for a Component Implementation-Equivalence Class may eliminate duplicate execution of implementation-specific activities within those configurations, but it does not replace interface-specific, channel-specific, integration, or end-to-end testing of the complete TOE.

Content and presentation elements:

ATE_IND.1.1C

The TOE shall be suitable for testing.

Evaluator action elements:

ATE_IND.1.1E

The evaluator *shall confirm* that the information provided meets all requirements for content and presentation of evidence.

ATE_IND.1.2E

The evaluator shall test a subset of the TSF to confirm that the TSF operates as specified.

Application Note: The evaluator should test the application on the most current fully patched version of the platform.

5.2.6 Class AVA: Vulnerability Assessment

For the current generation of this protection profile, the evaluation lab is expected to survey open sources to discover what vulnerabilities have been discovered in these types of products. In most cases, these vulnerabilities will require sophistication beyond that of a basic attacker. Until penetration tools are created and uniformly distributed to the evaluation labs, the evaluator will not be expected to test for these vulnerabilities in the TOE. The labs will be expected to comment on the likelihood of these vulnerabilities given the documentation provided by the vendor. This information will be used in the development of penetration testing tools and for the development of future protection profiles.

AVA_VAN.1 Vulnerability Survey (AVA_VAN.1)

Developer action elements:

AVA_VAN.1.1D

The developer shall provide the TOE for testing.

Content and presentation elements:

AVA_VAN.1.1C

The application shall be suitable for testing.

Application Note: Suitability for testing means not being obfuscated or packaged in such a way as to disrupt either static or dynamic analysis by the evaluator.

Evaluator action elements:

AVA_VAN.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AVA_VAN.1.2E

The evaluator shall perform a search of public domain sources to identify potential vulnerabilities in the TOE.

Application Note: Public domain sources include the Common Vulnerabilities and Exposures (CVE) dictionary for publicly known vulnerabilities. Public domain sources also include sites which provide free checking of files for viruses.

AVA_VAN.1.3E

The evaluator shall conduct penetration testing, based on the identified potential vulnerabilities, to determine that the TOE is resistant to attacks performed by an attacker possessing Basic attack potential.

Appendix A - Optional Requirements

As indicated in the introduction to this PP, the baseline requirements (those that must be performed by the TOE) are contained in the body of this PP. This appendix contains three other types of optional requirements:

The first type, defined in Appendix A.1 Strictly Optional Requirements, are strictly optional requirements. If the TOE meets any of these requirements the vendor is encouraged to claim the associated SERs in the ST, but doing so is not required in order to conform to this PP.

The second type, defined in Appendix A.2 Objective Requirements, are objective requirements. These describe security functionality that is not yet widely available in commercial technology. Objective requirements are not currently mandated by this PP, but will be mandated in the future. Adoption by vendors is encouraged, but claiming these SERs is not required in order to conform to this PP.

The third type, defined in Appendix A.3 Implementation-dependent Requirements, are Implementation-dependent requirements. If the TOE implements the product features associated with the listed SERs, either the SERs must be claimed or the product features must be disabled in the evaluated configuration.

A.1 Strictly Optional Requirements

A.1.1 Class ALC: Life-cycle Support

ALC_FLR.1 Basic Flaw Remediation (ALC_FLR.1)

This SAR is optional and may be claimed at the ST-Author's discretion.

Developer action elements:

ALC_FLR.1.1D

The developer shall document and provide flaw remediation procedures addressed to TOE developers.

Content and presentation elements:

ALC_FLR.1.1C

The flaw remediation procedures documentation shall describe the procedures used to track all reported security flaws in each release of the TOE.

ALC_FLR.1.2C

The flaw remediation procedures shall require that a description of the nature and effect of each security flaw be provided, as well as the status of finding a correction to that flaw.

ALC_FLR.1.3C

The flaw remediation procedures shall require that corrective actions be identified for each of the security flaws.

ALC_FLR.1.4C

The flaw remediation procedures documentation shall describe the methods used to provide flaw information, corrections and guidance on corrective actions to TOE users.

Evaluator action elements:

ALC_FLR.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_FLR.2 Flaw Reporting Procedures (ALC_FLR.2)

This SAR is optional and may be claimed at the ST-Author's discretion.

Developer action elements:

- ALC_FLR.2.1D The developer shall document and provide flaw remediation procedures addressed to TOE developers.
- ALC_FLR.2.2D The developer shall establish a procedure for accepting and acting upon all reports of security flaws and requests for corrections to those flaws.
- ALC_FLR.2.3D The developer shall provide flaw remediation guidance addressed to TOE users.

Content and presentation elements:

- ALC_FLR.2.1C The flaw remediation procedures documentation shall describe the procedures used to track all reported security flaws in each release of the TOE.
- ALC_FLR.2.2C The flaw remediation procedures shall require that a description of the nature and effect of each security flaw be provided, as well as the status of finding a correction to that flaw.
- ALC_FLR.2.3C The flaw remediation procedures shall require that corrective actions be identified for each of the security flaws.
- ALC_FLR.2.4C The flaw remediation procedures documentation shall describe the methods used to provide flaw information, corrections and guidance on corrective actions to TOE users.
- ALC_FLR.2.5C The flaw remediation procedures shall describe a means by which the developer receives from TOE users reports and enquiries of suspected security flaws in the TOE.
- ALC_FLR.2.6C The procedures for processing reported security flaws shall ensure that any reported flaws are remediated and the remediation procedures issued to TOE users.
- ALC_FLR.2.7C The procedures for processing reported security flaws shall provide safeguards that any corrections to these security flaws do not introduce any new flaws.
- ALC_FLR.2.8C The flaw remediation guidance shall describe a means by which TOE users report to the developer any suspected security flaws in the TOE.

Evaluator action elements:

- ALC_FLR.2.1E The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_FLR.3 Systematic Flaw Remediation (ALC_FLR.3)

This SAR is optional and may be claimed at the ST-Author's discretion.

Developer action elements:

- ALC_FLR.3.1D The developer shall document and provide flaw remediation procedures addressed to TOE developers.
- ALC_FLR.3.2D The developer shall establish a procedure for accepting and acting upon all reports of security flaws and requests for corrections to those flaws.
- ALC_FLR.3.3D The developer shall provide flaw remediation guidance addressed to TOE users.

Content and presentation elements:

ALC_FLR.3.1C	The flaw remediation procedures documentation shall describe the procedures used to track all reported security flaws in each release of the <u>TOE</u> .
ALC_FLR.3.2C	The flaw remediation procedures shall require that a description of the nature and effect of each security flaw be provided, as well as the status of finding a correction to that flaw.
ALC_FLR.3.3C	The flaw remediation procedures shall require that corrective actions be identified for each of the security flaws.
ALC_FLR.3.4C	The flaw remediation procedures documentation shall describe the methods used to provide flaw information, corrections and guidance on corrective actions to <u>TOE</u> users.
ALC_FLR.3.5C	The flaw remediation procedures shall describe a means by which the developer receives from <u>TOE</u> users reports and enquiries of suspected security flaws in the <u>TOE</u> .
ALC_FLR.3.6C	The flaw remediation procedures shall include a procedure requiring timely response and the automatic distribution of security flaw reports and the associated corrections to registered users who might be affected by the security flaw.
ALC_FLR.3.7C	The procedures for processing reported security flaws shall ensure that any reported flaws are remediated and the remediation procedures issued to <u>TOE</u> users.
ALC_FLR.3.8C	The procedures for processing reported security flaws shall provide safeguards that any corrections to these security flaws do not introduce any new flaws.
ALC_FLR.3.9C	The flaw remediation guidance shall describe a means by which <u>TOE</u> users report to the developer any suspected security flaws in the <u>TOE</u> .
ALC_FLR.3.10C	The flaw remediation guidance shall describe a means by which <u>TOE</u> users may register with the developer, to be eligible to receive security flaw reports and corrections.
ALC_FLR.3.11C	The flaw remediation guidance shall identify the specific points of contact for all reports and enquiries about security issues involving the <u>TOE</u> .

Evaluator action elements:

ALC_FLR.3.1E	The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.
--------------	--

A.2 Objective Requirements

A.2.1 Protection of the TSF (FPT)

FPT_API_EXT.2 Use of Supported Services and APIs

FPT_API_EXT.2.1	The application [selection, choose one of: <i>shall use platform-provided libraries, does not implement functionality</i>] for parsing [assignment: <i>list of formats parsed that are included in the <u>IANA MIME media types</u></i>]. Application Note: The <u>IANA MIME</u> types are listed at http://www.iana.org/assignments/media-types and include many image, audio, video, and content file formats.
-----------------	---

This requirement does not apply if providing parsing services is the purpose of the application.

FPT_IPC_EXT.1 Protection of Non-Network Local Inter-Process Communication

FPT_IPC_EXT.1.1

The application shall identify each non-network local inter-process communication mechanism used between TOE Component Instances and describe the [**assignment:** *base cPP requirements, platform access controls, object permissions, isolation mechanisms, configuration controls, or operational guidance*] that protect the communication path and restrict access to authorized TOE Component Instances.

Application Note: This objective applies to a TOE that uses non-network local inter-process communication between TOE Component Instances. Examples include named pipes, Unix domain sockets, shared memory, platform-brokered IPC, and equivalent mechanisms. It does not apply to in-process calls, direct local storage, or communication that uses a network interface or network protocol stack. The scope of a connection limited to the platform loopback interface is determined by [FTP_DIT_EXT.1](#); that connection is not covered by this objective.

FPT_IDV_EXT.1 Software Identification and Versions

FPT_IDV_EXT.1.1

The application shall be versioned with SWID tags that comply with minimum requirements from ISO/IEC 19770-2:2015.

Application Note: The use of a SWID tag to identify application software is a requirement for DoD IT based on DoD Instruction 8500.01 which requires the use of SCAP which includes SWID tags per the NIST standard.

Valid SWID tags must contain a SoftwareIdentity element and an Entity element as defined in the ISO/IEC 19770-2:2015 standard. SWID tags must be stored with a .swidtag file extensions as defined in the ISO/IEC 19770-2:2015.

A.3 Implementation-dependent Requirements

This PP does not define any Implementation-dependent requirements.

Appendix B - Selection-based Requirements

As indicated in the introduction to this PP, the baseline requirements (those that must be performed by the TOE or its underlying platform) are contained in the body of this PP. There are additional requirements based on selections in the body of the PP: if certain selections are made, then additional requirements below must be included.

B.1 Cryptographic Support (FCS)

FCS_CKM.1/AK Cryptographic Asymmetric Key Generation

The inclusion of this selection-based component depends upon selection in:

- [FCS_CKM_EXT.1.1](#)

FCS_CKM.1.1/AK

The application shall [selection:

- *invoke platform-provided functionality*
- *implement functionality*

] to generate **asymmetric** cryptographic keys in accordance with a specified cryptographic key generation algorithm [selection:

- *CNSA 2.0 Compliant Algorithms: [selection:*
 - **Leighton-Micali Signature Algorithm** using the parameter sets [selection: LMS_SHAKE_M24_H5, LMS_SHAKE_M24_H10, LMS_SHAKE_M24_H15, LMS_SHAKE_M24_H25, LMS_SHAKE_M32_H5, LMS_SHAKE_M32_H10, LMS_SHAKE_M32_H15, LMS_SHAKE_M32_H25, LMS_SHA256_M24_H5, LMS_SHA256_M24_H10, LMS_SHA256_M24_H15, LMS_SHA256_M24_H25, LMS_SHA256_M32_H5, LMS_SHA256_M32_H10, LMS_SHA256_M32_H15, LMS_SHA256_M32_H25] that meet the following [NIST SP 800-208, "Recommendation for Stateful Hash-Based Signature Schemes"]
 - **eXtended Merkle Signature Scheme Algorithm** using the parameter sets [selection: XMSS-SHA2_10_192, XMSS-SHA2_16_192, XMSS-SHA2_20_192, XMSS-SHA2_10_256, XMSS-SHA2_16_256, XMSS-SHA2_20_256, XMSS-SHAKE_10_192, XMSS-SHAKE_16_192, XMSS-SHAKE_20_192, XMSS-SHAKE_10_256, XMSS-SHAKE_16_256, XMSS-SHAKE_20_256] that meets the following: [NIST SP 800-208, "Recommendation for Stateful Hash-Based Signature Schemes"]
 - **Module-Lattice-Based Key-Encapsulation Mechanism Standard** using the parameter set ML-KEM-1024 that meets the following: [FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard]
 - **Module-Lattice-Based Digital Signature Standard** using the parameter set ML-DSA-87 that meets the following [FIPS 204, Module-Lattice-Based Digital Signature Standard]
- *CNSA 1.0 Compliant Algorithms: [selection:*
 - **[RSA schemes]** using cryptographic key sizes of [3072-bit or greater] that meet the following: [FIPS PUB 186-5, "Digital Signature Standard (DSS)," Appendix A.1]
 - **[ECC schemes]** using ["NIST curves" P-384 and [selection: P-521, no other curves]] that meet the following: [FIPS PUB 186-5, "Digital Signature Standard (DSS)," Appendix A.2]
 - **[FFC Schemes]** using ["safe-prime" groups] [selection: MODP-3072, MODP-4096, MODP-6144, MODP-8192, ffdhe-3072, ffdhe-4096, ffdhe-6144, ffdhe-8192] that meet the following: [NIST Special Publication 800-56A Revision 3, "Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography" and [selection: RFC 3526, RFC 7919]]

]

].

Application Note: The ST should claim all key generation schemes used for key establishment and entity authentication. When key generation is used for key establishment, the schemes in [FCS_CKM.2.1](#) and selected cryptographic protocols must match the selection. When key generation is used for entity authentication, the public key is expected to be associated with an X.509v3 certificate.

If the TOE acts as a receiver in the RSA key establishment scheme, the TOE does not need to implement RSA key generation.

Note that ML-DSA and ML-KEM are not usable in any functions at the time of initial publication, they are added to this requirement in support of future protocol updates. As support is expanded for CNSA 2.0, CNSA 1.0 will be removed as an selection in a future update.

FCS_CKM.1/SK Cryptographic Symmetric Key Generation

The inclusion of this selection-based component depends upon selection in:

- [FCS_COP.1.1/SKC](#)

FCS_CKM.1.1/SK

The **application** shall [**selection:** *invoke platform-provided functionality, implement functionality*] to generate **symmetric** cryptographic keys **using a Random Bit Generator as specified in [FCS_RBG_EXT.1](#)** and specified cryptographic key sizes *256-bit*

Application Note: This requirement is dependent on selecting any AES selection in [FCS_COP.1.1/SKC](#). Symmetric keys may be used to generate keys along the key chain.

FCS_CKM.2 Cryptographic Key Establishment

The inclusion of this selection-based component depends upon selection in:

- [FTP_DIT_EXT.1.1](#)

FCS_CKM.2.1

The application shall [**selection:** *invoke platform-provided functionality, implement functionality*] to perform cryptographic key establishment in accordance with a specified cryptographic key establishment method:

[**selection:**

- *CNSA 2.0 Compliant Algorithm:*
Module-Lattice-Based Key-Encapsulation Mechanism Standard using the parameter set *ML-KEM-1024* that meets the following: [[FIPS](#) 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard]
- *CNSA 1.0 Compliant Algorithms: [**selection:***
 - *[RSA-based key establishment schemes] that meet the following: [[NIST](#) Special Publication 800-56B, “Recommendation for Pair-Wise Key Establishment Schemes Using Integer Factorization Cryptography”]*
 - *[Elliptic curve-based key establishment schemes] that meets the following: [[NIST](#) Special Publication 800-56A, “Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography”]*
 - *[FFC Schemes using “safe-prime” groups] that meet the following: ‘[NIST](#) Special Publication 800-56A Revision 3, “Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography” and [**selection:** [RFC](#) 3526, [RFC](#) 7919]*

]

]

.

Application Note: The ST author shall select all key establishment schemes used for the selected cryptographic protocols. TLS requires cipher suites that use RSA-based key establishment schemes.

The RSA-based key establishment schemes are described in Section 9 of NIST SP 800-56B; however, Section 9 relies on implementation of other sections in SP 800-56B. If the TOE acts as a receiver in the RSA key establishment scheme, the TOE does not need to implement RSA key generation.

The elliptic curves used for the key establishment scheme shall correlate with the curves specified in [FCS_CKM.1.1/AK](#).

The domain parameters used for the finite field-based key establishment scheme are specified by the key generation according to [FCS_CKM.1.1/AK](#).

As support is expanded for CNSA 2.0, CNSA 1.0 will be removed as an selection in a future update.

FCS_COP.1/Hash Cryptographic Operation - Hashing

The inclusion of this selection-based component depends upon selection in:

- [FTP_DIT_EXT.1.1](#)

FCS_COP.1.1/Hash

The **application** shall perform [*cryptographic hashing services*] in accordance with a specified cryptographic algorithm [**selection**:

- *SHA-256*
- *SHA-384*
- *SHA-512*

] and **message digest** sizes [**selection**:

- *256*
- *384*
- *512*

] **bits** that meet the following: [*FIPS Pub 180-4, "Secure Hash Standard"*].

Application Note: This is dependent on implementing cryptographic functionality, as in [FTP_DIT_EXT.1](#).

The intent of this requirement is to specify the hashing function. The hash selection must support the message digest size selection.

FCS_COP.1/KeyedHash Cryptographic Operation - Keyed-Hash Message Authentication

The inclusion of this selection-based component depends upon selection in:

- [FTP_DIT_EXT.1.1](#)

FCS_COP.1.1/KeyedHash

The **application** shall perform [*keyed-hash message authentication*] in accordance with a specified cryptographic algorithm [**selection**:

- *HMAC-SHA-256*
- *HMAC-SHA-384*
- *HMAC-SHA-512*

] **with** key sizes [**assignment**: *key size (in bits) used in HMAC*] **and** **message digest sizes** [**selection**: *256, 384, 512*] **bits** that meet the following: [*FIPS Pub 198-1, "The Keyed-Hash Message Authentication Code," and FIPS Pub 180-4, "Secure Hash Standard"*].

Application Note: This is dependent on implementing cryptographic functionality, as in [FTP_DIT_EXT.1](#).

The intent of this requirement is to specify the keyed-hash message authentication function used for key establishment purposes for the various cryptographic protocols used by the application (e.g., trusted channel). The hash selection must support the message digest size selection.

FCS_COP.1/SigGen Cryptographic Operation - Signing Generation

The inclusion of this selection-based component depends upon selection in:

- [FTP_DIT_EXT.1.1](#)

FCS_COP.1.1/SigGen

The **application** shall perform [*cryptographic signature services (generation)*] in accordance with a specified cryptographic algorithm [**selection**]:

- *CNSA 2.0 Compliant Algorithm:*
Module-Lattice-Based Digital Signature Standard using the parameter set ML-DSA-87 that meets the following [FIPS 204, Module-Lattice-Based Digital Signature Standard]
- *CNSA 1.0 Compliant Algorithms: [selection]:*
 - **RSA schemes** using cryptographic key sizes of [3072-bit or greater] that meet the following: [FIPS PUB 186-5, "Digital Signature Standard (DSS)," Section 5]
 - **ECDSA schemes** using ["NIST curves" [selection: P-384, P-521]] that meet the following: [FIPS PUB 186-5, "Digital Signature Standard (DSS)," Section 6]

]

].

Application Note: This is dependent on implementing cryptographic functionality, as in [FTP_DIT_EXT.1](#).

The ST author should choose the algorithm implemented to perform digital signatures; if more than one algorithm is available, this requirement should be iterated to specify the functionality. For the algorithm chosen, the ST author should make the appropriate assignments/selections to specify the parameters that are implemented for that algorithm.

Note ML-DSA is not able to be used in any functions at the time of publication, it is being added for future support. As support is expanded for CNSA 2.0, CNSA 1.0 will be removed as an selection in a future update.

FCS_COP.1/SigVer Cryptographic Operation - Signing Verification

The inclusion of this selection-based component depends upon selection in:

- [FPT_TUD_EXT.2.3](#),
- [FTP_DIT_EXT.1.1](#)

FCS_COP.1.1/SigVer

The **application** shall perform [*cryptographic signature services (verification)*] in accordance with a specified cryptographic algorithm [**selection**]:

- *CNSA 2.0 Compliant Algorithms: [selection]:*
 - **Leighton-Micali Signature Algorithm** for verification using cryptographic key sizes of [selection: 192, 256] bits that meet the following [NIST.SP.800-208, "Recommendation for Stateful Hash-Based Signature Schemes"]
 - **eXtended Merkle Signature Scheme Algorithm** for verification using cryptographic key sizes of [selection: 192, 256] bits that meets the following: [NIST.SP.800-208, "Recommendation for Stateful Hash-Based Signature Schemes"]
 - **Module-Lattice-Based Digital Signature Standard** using the parameter set ML-DSA-87 that meets the following [FIPS 204, Module-Lattice-Based Digital Signature Standard]

]

- *CNSA 1.0 Compliant Algorithms: [selection]:*
 - **RSA schemes** using cryptographic key sizes of [3072-bit or greater] that meet the following: [FIPS PUB 186-5, "Digital Signature Standard (DSS)," Section 5]
 - **ECDSA schemes** using ["NIST curves" [selection: P-384, P-521]] that meet the following: [FIPS PUB 186-5, "Digital Signature Standard (DSS)," Section 6]

]

].

Application Note: This is dependent on implementing cryptographic functionality, as in [FTP_DIT_EXT.1](#).

The ST author should choose the algorithm implemented to perform digital signatures; if more than one algorithm is available, this requirement should be iterated to specify the functionality. For the algorithm chosen, the ST author should make the appropriate assignments/selections to specify the parameters that are implemented for that algorithm.

Note ML-DSA is not able to be used in any functions at the time of publication, it is being added for future support. As support is expanded for CNSA 2.0, CNSA 1.0 will be removed as an selection in a

future update.

If LMS or XMSS is claimed, then [FCS_COP.1/XOF](#) must also be claimed when SHAKE is utilized.

FCS_COP.1/XOF Cryptographic Operation - Extendable-Output Function

The inclusion of this selection-based component depends upon selection in:

- [FPT_TUD_EXT.2.3](#)

FCS_COP.1.1/XOF

The application shall perform [*extendable-output function*] in accordance with a specified cryptographic algorithm **SHAKE** and parameters **SHAKE256** that meet the following: [*NIST FIPS PUB 202 Section 6.2 [SHAKE]*].

Application Note: In accordance with CNSA 2.0, SHAKE is permitted to be used only as a component of LMS or XMSS. Therefore this component is claimed only if LMS or XMSS is claimed in [FCS_COP.1/SigVer](#) and they use SHAKE.

FCS_COP.1/SKC Cryptographic Operation - Encryption/Decryption

The inclusion of this selection-based component depends upon selection in:

- [FCS_STO_EXT.1.1](#),
- [FTP_DIT_EXT.1.1](#)

FCS_COP.1.1/SKC

The **application** shall [**selection:** *perform, invoke the platform to perform*] [*encryption and decryption*] in accordance with a specified cryptographic algorithm [**selection:**

- *AES-CBC (as defined in NIST SP. 800-38A) mode*
- *AES-GCM (as defined in NIST SP. 800-38D) mode*
- *AES-XTS (as defined in NIST SP. 800-38E) mode*
- *AES-CCM (as defined in NIST SP. 800-38C) mode*
- *AES-CTR (as defined in NIST SP. 800-38A) mode*

] and cryptographic key size of [256-bits].

Application Note: This is dependent on implementing cryptographic functionality, as in [FTP_DIT_EXT.1](#).

For the selection, the **ST** author should choose the mode or modes in which **AES** operates.

It is expected that symmetric keys will be generated or imported by the **TSE** as a dependency on this function, so [FCS_CKM.1/SK](#) must be claimed when this **SER** is claimed. [FCS_SNI_EXT.1](#) must also be claimed to define what, if any, salts the cryptographic algorithm implementation uses.

FCS_HTTPS_EXT.1 HTTPS Protocol

The inclusion of this selection-based component depends upon selection in:

- [FTP_DIT_EXT.1.1](#)

FCS_HTTPS_EXT.1.1

The application shall implement the **HTTPS** protocol as a [**selection:** *client, server, server with mutual authentication*] that complies with **REC 2818**.

FCS_HTTPS_EXT.1.2

The application shall implement **HTTPS** using **TLS** as defined in the Functional Package for **TLS**.

Application Note: This **SER** is claimed when [FTP_DIT_EXT.1](#) specifies the **TOE**'s use of **HTTPS** as a trusted protocol. If the **TSE** implements **HTTPS** as a client, or if it implements **HTTPS** as a server with mutual authentication, then [FCS_HTTPS_EXT.2](#) must be claimed to specify the **TSE**'s behavior when presented with an invalid peer certificate.

FCS_HTTPS_EXT.2 HTTPS Support for Authentication

The inclusion of this selection-based component depends upon selection in:

- [FCS_HTTPS_EXT.1.1](#),
- [FTP_DIT_EXT.1.1](#)

FCS_HTTPS_EXT.2.1

The application shall [**selection:** *establish the connection, not establish the connection, establish or not establish the connection based on an administrative or user setting*] if the peer certificate is deemed invalid when attempting to establish a ~~HTTPS~~ connection.

Application Note: This ~~SFR~~ is claimed when the ~~TOE~~ implements ~~HTTPS~~ as a client, or when the ~~TOE~~ implements ~~HTTPS~~ as a server with support for mutual authentication.

FCS_PBKDF_EXT.1 Password Conditioning

The inclusion of this selection-based component depends upon selection in:

- [FCS_STO_EXT.1.1](#)

FCS_PBKDF_EXT.1.1

The application shall condition passwords/passphrases with [**assignment:** *Password-based Key Derivation Functions*] in accordance with a specified cryptographic algorithm as specified in FCS_COP.1 /**KeyedHash**, with [**selection:** [**assignment:** *positive integer between 1,000 and 9,999*], [**assignment:** *positive integer between 1,0000 and 199,999*], [**assignment:** *positive integer greater than 200,000*]] iterations, and output size of [**assignment:** *positive integer of 256 or greater*] bits that meet the following [~~NIST SP 800-132~~].

FCS_PBKDF_EXT.1.2

The ~~TSE~~ shall generate salts in accordance with [FCS_SNI_EXT.1](#) and with entropy corresponding to the security strength selected for PBKDF in [FCS_PBKDF_EXT.1](#).

Application Note: This should be included if selected in [FCS_STO_EXT.1](#).

Conditioning can be performed using one of the identified hash functions or the process described in ~~NIST SP 800-132~~; the method used is selected by the ~~ST~~ Author. ~~SP 800-132~~ requires the use of a pseudorandom function (PRF) consisting of ~~HMAC~~ with an approved hash function. The ~~ST~~ author selects the hash function used, including the appropriate requirements for ~~HMAC~~ and the hash function.

Appendix A of SP800-132 recommends setting the number of iterations as high as can be tolerated for the environment, while maintaining acceptable performance. For unconstrained environments, this could be 200,000 or much higher. The larger the iteration count, the greater protection is against a password recovery attack due to the increase computation needed to a derive a key. This value is expected to increase to a minimum of 10,000 in a future iteration based on ~~NIST SP 800-63~~.

FCS_RBG.1 Random Bit Generation (RBG)

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG_EXT.1.1](#)

FCS_RBG.1.1

The ~~TSE~~ shall perform deterministic random bit generation services using [**selection:**

- *Hash_DRBG (any)*
- *HMAC_DRBG (any)*
- *CTR_DRBG (~~AES~~)*

] in accordance with [~~NIST SP 800-90A~~] after initialization with a seed.

Application Note: ~~NIST SP 800-90A~~ contains three different methods of generating random numbers; each of these, in turn, depends on underlying cryptographic primitives (hash functions/ciphers). The ~~ST~~ author will select the function used and include the specific underlying cryptographic primitives used in the requirement or in the ~~TSS~~. While any of the identified hash functions (~~SHA-224~~, ~~SHA-256~~, ~~SHA-384~~, ~~SHA-512~~) are allowed for Hash_DRBG or HMAC_DRBG, only ~~AES~~-based implementations for CTR_DRBG are allowed.

FCS_RBG.1.2

The **TSE** shall use a [**selection**: *TSE noise source* [**assignment**: *name of noise source*], **multiple TSE noise sources** [**assignment**: *names of noise sources*], *TSE interface for seeding*] for initialized seeding.

Application Note: For the selection in this requirement, the **ST** author selects "TSE noise source" if a single noise source is used as input to the **DRBG**. The **ST** author selects "multiple TSE noise sources" if a seed is formed from a combination of two or more noise sources within the **TOE** boundary. If the **TSE** implements two or more separate DRBGs that are seeded in separate manners, this **SER** should be iterated for each **DRBG**. If multiple distinct noise sources exist such that each **DRBG** only uses one of them, then each iteration would select "TSE noise source"; "multiple TSE noise sources" is only selected if a single **DRBG** uses multiple noise sources for its seed. The **ST** author selects "TSE interface for seeding" if noise source data is generated outside the **TOE** boundary.

If "TSE noise source" is selected, [FCS_RBG.3](#) must be claimed.

If "multiple TSE noise sources" is selected, [FCS_RBG.4](#) and [FCS_RBG.5](#) must be claimed.

If "TSE interface for seeding" is selected, [FCS_RBG.2](#) must be claimed.

FCS_RBG.1.3

The **TSE** shall update the **RBG** state by [**selection**: *reseeding, uninstantiating and reinstantiating*] using a [**selection**: *TSE noise source* [**assignment**: *name of noise source*], *TSE interface for seeding*] in the following situations: [**selection**:

- *on demand*
- *on the condition*: [**assignment**: *condition*]
- *after* [**assignment**: *time*]

] in accordance with [**assignment**: *list of standards*].

FCS_RBG.2 Random Bit Generation (External Seeding)

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG.1.2](#)

FCS_RBG.2.1

The **TSE** shall be able to accept a minimum input of [**assignment**: *minimum input length, in bits, greater than zero*] from a **TSE** interface for the purpose of seeding.

Application Note: This requirement is claimed when a **DRBG** is seeded with entropy from one or more noise sources that are outside the **TOE** boundary. Typically the entropy produced by an environmental noise source is conditioned such that the input length has full entropy and is therefore usable as the seed. However, if this is not the case, it should be noted what the minimum entropy rate of the noise source is so that the **TSE** can collect a sufficiently large sample of noise data to be conditioned into a seed value.

FCS_RBG.3 Random Bit Generation (Internal Seeding - Single Source)

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG.1.2](#)

FCS_RBG.3.1

The **TSE** shall be able to seed the **RBG** using a **TSE** software-based noise source with a minimum of [**assignment**: *number of bits*] bits of min-entropy.

Application Note: This requirement is claimed when a **DRBG** is seeded with entropy from a single noise source that is within the **TOE** boundary. Min-entropy should be expressed as a ratio of entropy bits to sampled bits so that the total amount of data needed to ensure full entropy is known, as well as the conditioning function by which that data is reduced in size to the seed.

FCS_RBG.4 Random Bit Generation (Internal Seeding - Multiple Sources)

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG.1.2](#)

FCS_RBG.4.1

The ~~T.SF~~ shall be able to seed the ~~RBG~~ using [assignment: *number*] ~~T.SF~~ software-based noise source(s).

Application Note: This requirement is claimed when a ~~DRBG~~ is seeded with entropy from multiple noise sources that are within the ~~TOE~~ boundary. [FCS_RBG.5](#) defines the mechanism by which these sources are combined to ensure sufficient minimum entropy.

FCS_RBG.5 Random Bit Generation (Combining Noise Sources)

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG.1.2](#)

FCS_RBG.5.1

The ~~T.SF~~ shall [assignment: *combining operation*] [selection: *output from T.SF noise source(s), input from T.SF interface(s) for seeding*] to create the entropy input into the derivation function as defined in [assignment: *list of standards*], resulting in a minimum of [assignment: *number of bits*] bits of min-entropy.

Application Note: Examples of typical combining operations include, but are not limited to, XORing or hashing.

FCS_SNI_EXT.1 Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation)

The inclusion of this selection-based component depends upon selection in:

- [FCS_COP.1.1/SKC](#),
- [FCS_STO_EXT.1.1](#)

FCS_SNI_EXT.1.1

The application shall [selection: *use no salts, use salts that are generated by a ~~DRBG~~ as specified in [FCS_RBG_EXT.1](#)*]

FCS_SNI_EXT.1.2

The application shall use [selection: *no nonces, unique nonces with a minimum size of [64] bits.*]

FCS_SNI_EXT.1.3

The application shall [selection:

- *use no IVs*
- *create IVs in the following manner [selection:*
 - *CBC: IVs shall be non-repeating and unpredictable;*
 - *CCM: Nonce shall be non-repeating;*
 - *CTR: "Initial Counter" shall be non-repeating. No counter value shall be repeated across multiple messages with the same secret key.*
 - *XTS: No IV. Tweak values shall be non-negative integers, assigned consecutively, and starting at an arbitrary non-negative integer;*
 - *GCM: IV shall be non-repeating. The number of invocations of GCM shall not exceed 2^{32} for a given secret key. The IV constructed using one of two allowed construction methods given in Section 8.2 of NIST SP 800-38D.*

]

]

Application Note: This requirement ensures that salts, nonces, and initialization vectors are properly implemented. If the application is implementing a salt, nonce, or initialization vector they must select the corresponding selection. If the platform implements these functions, the corresponding "use no..." options are selected.

This requirement is dependent on selecting "implement functionality to securely store..." in [FCS_STO_EXT.1.1](#) or any ~~AES~~ selection in [FCS_COP.1.1/SKC](#).

B.2 Protection of the TSF (FPT)

FPT_FLS.1 Failure with Preservation of Secure State

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG_EXT.1.1](#)

FPT_FLS.1.1

The TSF shall preserve a secure state when the following types of failures occur: [*DRBG self-test failure*].

Application Note: The intent of this requirement is to ensure that cryptographic services requiring random bit generation cannot be performed if a failure of a self-test defined in [FPT_TST.1](#) occurs.

FPT_TST.1 TSF Self-Testing

The inclusion of this selection-based component depends upon selection in:

- [FCS_RBG_EXT.1.1](#)

FPT_TST.1.1

The TSF shall run a suite of the following self-tests [**selection:** *during initial start-up, periodically during normal operation, at the request of the authorized user, at the conditions* [**assignment:** *conditions under which self-test should occur*]] to demonstrate the correct operation of [*TSF DRBG specified in [FCS_RBG.1](#)*].

FPT_TST.1.2

The TSF shall provide authorized users with the capability to verify the integrity of [*DRBG seed/output data*].

FPT_TST.1.3

The TSF shall provide authorized users with the capability to verify the integrity of [*TSF DRBG specified in [FCS_RBG.1](#)*].

Application Note: This SER is a required dependency of [FCS_RBG.1](#). It is intended to require that any DRBG implemented by the TOE undergo health testing to ensure that the random bit generation functionality has not been degraded. If the TSF supports multiple DRBGs, this SER should be iterated to describe the self-test behavior for each.

FPT_TUD_EXT.2 Integrity for Installation and Update

The inclusion of this selection-based component depends upon selection in:

- [FPT_TUD_EXT.1.3](#),
- [FPT_TUD_EXT.1.5](#)

FPT_TUD_EXT.2.1

The application shall be distributed using [**selection:** *the format of the platform-supported package manager, a container image*].

FPT_TUD_EXT.2.2

The application shall be packaged such that its removal results in the deletion of all traces of the application, with the exception of configuration settings, output files, and audit/log events.

Application Note: Application software bundled with the system/firmware image are not subject to this requirement if the user is unable to remove the application through means provided by the OS.

FPT_TUD_EXT.2.3

The application installation package shall be digitally signed such that [**selection:**

- *its platform can cryptographically verify them prior to installation.*
- *the application can verify them using* [**selection:** *Leighton-Micali Signature., eXtended Merkle Signature Scheme.*]

]

Application Note: The specifics of the verification of installation packages involves requirements on the platform (and not the application), so these are not fully specified here.

If "**Leighton-Micali Signature**" or "**eXtended Merkle Signature Scheme**" is selected, the corresponding selection must be made in [FCS_COP.1/SigVer](#).

Appendix C - Extended Component Definitions

This appendix contains the definitions for all extended requirements specified in the PP.

C.1 Extended Components Table

All extended components specified in the PP are listed in this table:

Table 3: Extended Component Definitions

Functional Class	Functional Components
Cryptographic Support (FCS)	FCS_CKM_EXT Cryptographic Key Management FCS_HTTPS_EXT HTTPS Protocol FCS_PBKDF_EXT Password Conditioning FCS_RBG_EXT Random Bit Generation FCS_STO_EXT Storage of Credentials
Privacy (FPR)	FPR_ANO_EXT User Consent for Transmission of Personally Identifiable Information
Protection of the TSE (FPT)	FPT_AEX_EXT Anti-Exploitation Capabilities FPT_API_EXT Use of Supported Services and APIs FPT_IDV_EXT Software Identification and Versions FPT_IPC_EXT Protection of Non-Network Local Inter-Process Communication FPT_LIB_EXT TSE Use of Third Party Libraries FPT_TUD_EXT Trusted Updates
Security Management (FMT)	FMT_CFG_EXT Secure by Default Configuration FMT_MEC_EXT Supported Configuration Mechanism
Trusted Path/Channels (FTP)	FTP_DIT_EXT Protection of Data in Transit
User Data Protection (FDP)	FDP_DAR_EXT Data-at-Rest Encryption FDP_DEC_EXT Access to Platform Resources FDP_NET_EXT Network Communications

C.2 Extended Component Definitions

C.2.1 Cryptographic Support (FCS)

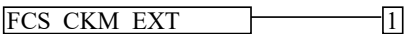
This PP defines the following extended components as part of the FCS class originally defined by CC Part 2:

C.2.1.1 FCS_CKM_EXT Cryptographic Key Management

Family Behavior

This family defines requirements for management of cryptographic keys that are not addressed by FCS_CKM in CC Part 2.

Component Leveling



FCS_CKM_EXT.1, Cryptographic Key Generation Services, requires the TSE to specify whether asymmetric key generation is implemented by the TSE, invoked from the operational environment, or not used by the TOE.

Management: FCS_CKM_EXT.1

No specific management functions are identified.

Audit: FCS_CKM_EXT.1

There are no auditable events foreseen.

FCS_CKM_EXT.1 Cryptographic Key Generation Services

Hierarchical to: No other components.

Dependencies to: No dependencies.

FCS_CKM_EXT.1.1

The application shall [selection:

- *generate no asymmetric cryptographic keys*
- *invoke platform-provided functionality for asymmetric key generation*
- *implement asymmetric key generation*

].

C.2.1.2 FCS_HTTPS_EXT HTTPS Protocol

Family Behavior

This family defines requirements for implementation of the HTTPS protocol.

Component Leveling



FCS_HTTPS_EXT.1, HTTPS Protocol, defines the capability of the TOE to implement HTTPS.

FCS_HTTPS_EXT.2, HTTPS Support for Authentication, defines the TSF's response when an invalid certificate is presented as part of HTTPS connection establishment.

Management: FCS_HTTPS_EXT.1

No specific management functions are identified.

Audit: FCS_HTTPS_EXT.1

There are no auditable events foreseen.

FCS_HTTPS_EXT.1 HTTPS Protocol

Hierarchical to: No other components.

Dependencies to: FCS_TLS_EXT.1 TLS Protocol

FCS_HTTPS_EXT.1.1

The application shall implement the HTTPS protocol as a [selection: *client*, *server*, *server with mutual authentication*] that complies with REC 2818.

FCS_HTTPS_EXT.1.2

The application shall implement HTTPS using TLS as defined in the Functional Package for TLS.

Management: FCS_HTTPS_EXT.2

The following actions could be considered for the management functions in FMT:

- Configuration of HTTPS behavior in response to receipt of invalid certificate.

Audit: FCS_HTTPS_EXT.2

There are no auditable events foreseen.

FCS_HTTPS_EXT.2 HTTPS Support for Authentication

Hierarchical to: No other components.

Dependencies to: [FCS_HTTPS_EXT.1](#) HTTPS Protocol
[FIA_X509_EXT.1](#) X.509 Certificate Validation

FCS_HTTPS_EXT.2.1

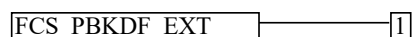
The application shall [**selection:** *establish the connection, not establish the connection, establish or not establish the connection based on an administrative or user setting*] if the peer certificate is deemed invalid when attempting to establish a ~~HTTPS~~ connection.

C.2.1.3 FCS_PBKDF_EXT Password Conditioning

Family Behavior

This family defines requirements for implementation of password-based key derivation functions.

Component Leveling



[FCS_PBKDF_EXT.1](#), Password Conditioning, defines the capability of the ~~TQE~~ to implement PBKDF2 for key derivation.

Management: FCS_PBKDF_EXT.1

No specific management functions are identified.

Audit: FCS_PBKDF_EXT.1

There are no auditable events foreseen.

FCS_PBKDF_EXT.1 Password Conditioning

Hierarchical to: No other components.

Dependencies to: [FCS_COP.1](#) Cryptographic Operation
[FCS_RBG_EXT.1](#) Random Bit Generation Services

FCS_PBKDF_EXT.1.1

The application shall condition passwords/passphrases with [**assignment:** *Password-based Key Derivation Functions*] in accordance with a specified cryptographic algorithm as specified in [FCS_COP.1](#), with [**assignment:** *positive integer of 1,000 or greater*] iterations, and output cryptographic key sizes [**assignment:** *positive integer of 256 or greater*] bits that meet the following [**assignment:** *applicable standard*].

FCS_PBKDF_EXT.1.2

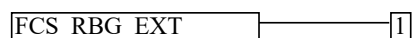
The ~~TSE~~ shall generate salts in accordance with [FCS_SNI_EXT.1](#) and with entropy corresponding to the security strength selected for PBKDF in [FCS_PBKDF_EXT.1](#).

C.2.1.4 FCS_RBG_EXT Random Bit Generation

Family Behavior

This family defines requirements for the generation of random bits.

Component Leveling



[FCS_RBG_EXT.1](#), Random Bit Generation Services, requires the [TSE](#) to specify whether random bit generation is implemented by the [TSE](#), invoked from the operational environment, or not used by the [TOE](#).

Management: FCS_RBG_EXT.1

No specific management functions are identified.

Audit: FCS_RBG_EXT.1

There are no auditable events foreseen.

FCS_RBG_EXT.1 Random Bit Generation Services

Hierarchical to: No other components.

Dependencies to: No dependencies.

FCS_RBG_EXT.1.1

The application shall [selection:

- *use no ~~DRBG~~ functionality*
- *invoke platform-provided ~~DRBG~~ functionality*
- *implement ~~DRBG~~ functionality*

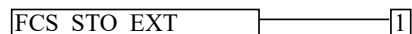
] for its cryptographic operations.

C.2.1.5 FCS_STO_EXT Storage of Credentials

Family Behavior

This family defines requirements for the secure storage of credential data.

Component Leveling



[FCS_STO_EXT.1](#), Storage of Credentials, requires the application to define how to store credentials to non-volatile memory.

Management: FCS_STO_EXT.1

No specific management functions are identified.

Audit: FCS_STO_EXT.1

There are no auditable events foreseen.

FCS_STO_EXT.1 Storage of Credentials

Hierarchical to: No other components.

Dependencies to: No dependencies.

FCS_STO_EXT.1.1

The application shall [selection:

- *not store any credentials*
- *invoke the functionality provided by the platform to securely store [assignment: list of credentials]*
- *implement functionality to securely store [assignment: list of credentials] according to [assignment: cryptographic mechanisms]*

] to non-volatile memory.

C.2.2 Privacy (FPR)

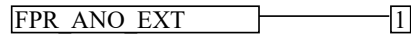
This PP defines the following extended components as part of the FPR class originally defined by CC Part 2:

C.2.2.1 FPR_ANO_EXT User Consent for Transmission of Personally Identifiable Information

Family Behavior

This family defines requirements for anonymity that are not covered by the Part 2 family FPR_ANO.

Component Leveling



[FPR_ANO_EXT.1](#), User Consent for Transmission of Personally Identifiable Information, requires the TSF to transmit personally identifiable information only with explicit approval.

Management: FPR_ANO_EXT.1

The following action could be considered for the management functions in FMT:

- Enabling and disabling the transmission of any PII.

Audit: FPR_ANO_EXT.1

There are no auditable events foreseen.

FPR_ANO_EXT.1 User Consent for Transmission of Personally Identifiable Information

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPR_ANO_EXT.1.1

The application shall [selection, choose one of:

- *not use PII*
- *not transmit PII over a network*
- *require user approval before executing [assignment: list of functions that transmit PII over a network]*

].

C.2.3 Protection of the TSF (FPT)

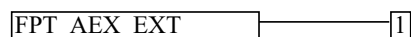
This PP defines the following extended components as part of the FPT class originally defined by CC Part 2:

C.2.3.1 FPT_AEX_EXT Anti-Exploitation Capabilities

Family Behavior

This family defines requirements for protecting against common types of software exploitation techniques.

Component Leveling



[FPT_AEX_EXT.1](#), Anti-Exploitation Capabilities, requires the application to implement functionality that protects against common software exploits.

Management: FPT_AEX_EXT.1

No specific management functions are identified.

Audit: FPT_AEX_EXT.1

There are no auditable events foreseen.

FPT_AEX_EXT.1 Anti-Exploitation Capabilities

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_AEX_EXT.1.1

The application shall not request to map memory at an explicit address except for [assignment: *list of explicit exceptions*].

FPT_AEX_EXT.1.2

The application shall [selection, choose one of:

- *not allocate any memory region with both write and execute permissions*
- *allocate memory regions with write and execute permissions for only [assignment: *list of functions performing just-in-time compilation*]*

].

FPT_AEX_EXT.1.3

The application shall be compatible with security features provided by the platform vendor.

FPT_AEX_EXT.1.4

The application shall not write user-modifiable files to directories that contain executable files unless explicitly directed by the user to do so.

FPT_AEX_EXT.1.5

The application shall be built with stack-based buffer overflow protection enabled.

C.2.3.2 FPT_API_EXT Use of Supported Services and APIs

Family Behavior

This family defines requirements for specifying the environmental APIs used by the TOE.

Component Leveling



[FPT_API_EXT.1](#), Use of Supported Services and APIs, requires the application to use only documented platform APIs.

[FPT_API_EXT.2](#), Use of Supported Services and APIs, requires the application to implement media parsing in a specified manner.

Management: FPT_API_EXT.1

No specific management functions are identified.

Audit: FPT_API_EXT.1

There are no auditable events foreseen.

FPT_API_EXT.1 Use of Supported Services and APIs

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_API_EXT.1.1

The application shall use only documented platform APIs.

Management: FPT_API_EXT.2

No specific management functions are identified.

Audit: FPT_API_EXT.2

There are no auditable events foreseen.

FPT_API_EXT.2 Use of Supported Services and APIs

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_API_EXT.2.1

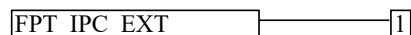
The application [selection, choose one of: *shall use platform-provided libraries, does not implement functionality*] for parsing [assignment: *list of formats parsed that are included in the IANA MIME media types*].

C.2.3.3 FPT_IPC_EXT Protection of Non-Network Local Inter-Process Communication

Family Behavior

This family defines objective requirements for identifying and describing the protection of non-network local inter-process communication between TOE Component Instances.

Component Leveling



[FPT_IPC_EXT.1](#), Protection of Non-Network Local Inter-Process Communication, requires the application to identify and describe the protection of non-network local inter-process communication between TOE Component Instances.

Management: FPT_IPC_EXT.1

No specific management functions are identified.

Audit: FPT_IPC_EXT.1

There are no auditable events foreseen.

FPT_IPC_EXT.1 Protection of Non-Network Local Inter-Process Communication

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_IPC_EXT.1.1

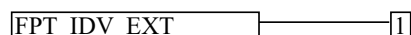
The application shall identify each non-network local inter-process communication mechanism used between TOE Component Instances and describe the [assignment: *base CP requirements, platform access controls, object permissions, isolation mechanisms, configuration controls, or operational guidance*] that protect the communication path and restrict access to authorized TOE Component Instances.

C.2.3.4 FPT_IDV_EXT Software Identification and Versions

Family Behavior

This family defines requirements for how the TOE version is identified.

Component Leveling



[FPT_IDV_EXT.1](#), Software Identification and Versions, requires the [TSF](#) to specify the versioning mechanism used.

Management: FPT_IDV_EXT.1

No specific management functions are identified.

Audit: FPT_IDV_EXT.1

There are no auditable events foreseen.

FPT_IDV_EXT.1 Software Identification and Versions

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_IDV_EXT.1.1

The application shall be versioned with [selection: *SWID tags that comply with minimum requirements from ISO/IEC 19770-2:2015* , [assignment: *other version information*]].

C.2.3.5 FPT_LIB_EXT TSF Use of Third Party Libraries

Family Behavior

This family defines requirements for identification of any third-party libraries used by the [TOE](#).

Component Leveling



[FPT_LIB_EXT.1](#), Use of Third Party Libraries, requires the [TOE](#) to identify the third party libraries that it uses.

Management: FPT_LIB_EXT.1

No specific management functions are identified.

Audit: FPT_LIB_EXT.1

There are no auditable events foreseen.

FPT_LIB_EXT.1 Use of Third Party Libraries

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_LIB_EXT.1.1

The application shall be packaged with only [assignment: *list of third-party libraries*].

C.2.3.6 FPT_TUD_EXT Trusted Updates

Family Behavior

This family defines requirements for applying updates to the [TOE](#).

Component Leveling



[FPT_TUD_EXT.1](#), Support for Trusted Updates, requires the [TSF](#) to specify how updates to it are acquired and verified.

[FPT_TUD_EXT.2](#), Integrity for Installation and Update, requires [TOE](#) updates to be packaged in a certain manner.

Management: FPT_TUD_EXT.1

No specific management functions are identified.

Audit: FPT_TUD_EXT.1

There are no auditable events foreseen.

FPT_TUD_EXT.1 Support for Trusted Updates

Hierarchical to: No other components.

Dependencies to: [FPT_IDV_EXT.1](#) Software Identification and Versions

FPT_TUD_EXT.1.1

The application shall [**selection:** *provide the ability, use platform-provided services*] to check for updates and patches to the application software.

FPT_TUD_EXT.1.2

The application shall [**selection:** *provide the ability, use platform-provided services*] to query the current version of the application software.

FPT_TUD_EXT.1.3

The application shall [**selection:**

- *perform trusted updates*
- *not download, modify, replace or update its own binary code*

].

FPT_TUD_EXT.1.4

Application updates shall be digitally signed such that the application platform can cryptographically verify them prior to installation.

FPT_TUD_EXT.1.5

The application is distributed [**selection:** *with the platform QS, as an additional software package to the platform QS*].

Management: FPT_TUD_EXT.2

No specific management functions are identified.

Audit: FPT_TUD_EXT.2

There are no auditable events foreseen.

FPT_TUD_EXT.2 Integrity for Installation and Update

Hierarchical to: No other components.

Dependencies to: [FPT_TUD_EXT.1](#) Integrity for Installation and Update

FPT_TUD_EXT.2.1

The application shall be distributed using [**selection:** *the format of the platform-supported package manager, a container image*].

FPT_TUD_EXT.2.2

The application shall be packaged such that its removal results in the deletion of all traces of the application, with the exception of configuration settings, output files, and audit/log events.

FPT_TUD_EXT.2.3

The application installation package shall be digitally signed such that [**selection:**

- its platform can cryptographically verify them prior to installation.
- the application can verify them using [**selection**: Leighton-Micali Signature., eXtended Merkle Signature Scheme.]

]

C.2.4 Security Management (FMT)

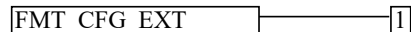
This PP defines the following extended components as part of the FMT class originally defined by CC Part 2:

C.2.4.1 FMT_CFG_EXT Secure by Default Configuration

Family Behavior

This family defines requirements for authorization to manage the behavior of the application.

Component Leveling



FMT_CFG_EXT.1, Secure by Default Configuration, requires the application to define how to set new credentials and protect the application from modification by unprivileged users.

Management: FMT_CFG_EXT.1

No specific management functions are identified.

Audit: FMT_CFG_EXT.1

There are no auditable events foreseen.

FMT_CFG_EXT.1 Secure by Default Configuration

Hierarchical to: No other components.

Dependencies to: No dependencies.

FMT_CFG_EXT.1.1

The application shall [**selection**: *not use credentials, use platform-provided credentials, provide only enough functionality to set new credentials when configured with default credentials or no credentials for application provided credentials*].

FMT_CFG_EXT.1.2

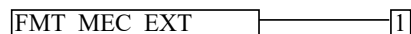
The application shall be configured by default with file permissions which protect the application binaries and data files from modification by normal unprivileged users.

C.2.4.2 FMT_MEC_EXT Supported Configuration Mechanism

Family Behavior

This family defines requirements for the TOE's use of mechanisms for the storage of configuration data.

Component Leveling



FMT_MEC_EXT.1, Supported Configuration Mechanism, requires the application to store configuration data either through the use of an appropriate environmental mechanism or through its own file encryption capability.

Management: FMT_MEC_EXT.1

No specific management functions are identified.

Audit: FMT_MEC_EXT.1

There are no auditable events foreseen.

FMT_MEC_EXT.1 Supported Configuration Mechanism

Hierarchical to: No other components.

Dependencies to: No dependencies.

FMT_MEC_EXT.1.1

The application shall [selection: *invoke the mechanisms recommended by the platform vendor for storing and setting configuration options, implement functionality to encrypt and store configuration options as defined by FDP_PRT_EXT.1 in the PP-Module for File Encryption*].

C.2.5 Trusted Path/Channels (FTP)

This PP defines the following extended components as part of the FTP class originally defined by CC Part 2:

C.2.5.1 FTP_DIT_EXT Protection of Data in Transit

Family Behavior

This family defines requirements for protecting data in transit.

Component Leveling



FTP_DIT_EXT.1, Protection of Data in Transit, requires the TSF to specify what data is transmitted outside the TOE or, where the TOE Component Instance peer is selected, over an in-scope network-mediated channel between TOE Component Instances, what protocol is used for data transmission, and whether the TSF implements this protocol or invokes an environmental interface to do so.

Management: FTP_DIT_EXT.1

No specific management functions are identified.

Audit: FTP_DIT_EXT.1

There are no auditable events foreseen.

FTP_DIT_EXT.1 Protection of Data in Transit

Hierarchical to: No other components.

Dependencies to: No dependencies.

FTP_DIT_EXT.1.1

The application shall [selection, choose one of:

- *not transmit any [selection, choose one of: data , sensitive data]*
- *encrypt all transmitted [selection, choose one of: sensitive data , data] with [assignment: trusted protocol] for [assignment: function(s)]*
- *invoke platform-provided functionality to encrypt all transmitted sensitive data with [assignment: trusted protocol] for [assignment: function(s)]*
- *invoke platform-provided functionality to encrypt all transmitted data with [assignment: trusted protocol] for [assignment: function(s)]*

] between itself and another [selection: *trusted IT product, TOE Component Instance*].

C.2.6 User Data Protection (FDP)

This PP defines the following extended components as part of the FDP class originally defined by CC Part 2:

C.2.6.1 FDP_DAR_EXT Data-at-Rest Encryption

Family Behavior

This family defines requirements for implementation of data-at-rest protection.

Component Leveling



[FDP_DAR_EXT.1](#), Encryption Of Sensitive Application Data, requires the application to be able to protect all data with a chosen method of encryption.

Management: FDP_DAR_EXT.1

No specific management functions are identified.

Audit: FDP_DAR_EXT.1

There are no auditable events foreseen.

FDP_DAR_EXT.1 Encryption Of Sensitive Application Data

Hierarchical to: No other components.

Dependencies to: No dependencies.

FDP_DAR_EXT.1.1

The application shall [selection:

- *leverage platform-provided functionality to encrypt sensitive data*
- *implement functionality to encrypt sensitive data as defined in the PP-Module for File Encryption*
- *protect sensitive data in accordance with [FCS_STO_EXT.1](#)*
- *not store any sensitive data*

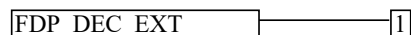
] in non-volatile memory.

C.2.6.2 FDP_DEC_EXT Access to Platform Resources

Family Behavior

This family defines requirements for accessing platform resources.

Component Leveling



[FDP_DEC_EXT.1](#), Access to Platform Resources, requires the application to restrict access to hardware sources and sensitive information repositories.

Management: FDP_DEC_EXT.1

The following action could be considered for the management functions in FMT:

- Enabling and disabling the transmission of any information describing the system's hardware, software, or configuration.

Audit: FDP_DEC_EXT.1

There are no auditable events foreseen.

FDP_DEC_EXT.1 Access to Platform Resources

Hierarchical to: No other components.

Dependencies to: FCS_TLS_EXT.1 TLS Protocol
FIA_X509_EXT.1 X.509 Certificate Validation

FDP_DEC_EXT.1.1

The application shall restrict its access to only [selection:

- *no hardware resources*

- *network connectivity*
- *camera*
- *microphone*
- *location services*
- *NFC*
- *USB*
- *Bluetooth*
- *[assignment: list of additional hardware resources]*

].

FDP_DEC_EXT.1.2

The application shall restrict its access to only [selection:

- *no sensitive information repositories*
- *address book*
- *calendar*
- *call lists*
- *system logs*
- *[assignment: list of additional sensitive information repositories]*

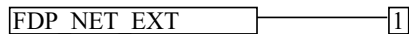
].

C.2.6.3 FDP_NET_EXT Network Communications

Family Behavior

This family defines requirements for the TOE's use of network connectivity.

Component Leveling



[FDP_NET_EXT.1](#), Network Communications, identifies the purpose for each network interface used by the TOE and how that interface is invoked.

Management: FDP_NET_EXT.1

No specific management functions are identified.

Audit: FDP_NET_EXT.1

There are no auditable events foreseen.

FDP_NET_EXT.1 Network Communications

Hierarchical to: No other components.

Dependencies to: No dependencies.

FDP_NET_EXT.1.1

The application shall restrict network communication to [selection:

- *no network communication*
- *user-initiated communication for [assignment: list of functions for which the user can initiate network communication]*
- *respond to [assignment: list of remotely initiated communication]*
- *[assignment: list of application-initiated network communication]*

].

Appendix D - Entropy Documentation and Assessment

This appendix describes the required supplementary information for the entropy source used by the TOE.

The documentation of the entropy source should be detailed enough that, after reading, the evaluator will thoroughly understand the entropy source and why it can be relied upon to provide sufficient entropy. This documentation should include multiple detailed sections: design description, entropy justification, operating conditions, and health testing. This documentation is not required to be part of the TSS.

D.1 Design Description

Documentation shall include the design of the entropy source as a whole, including the interaction of all entropy source components. Any information that can be shared regarding the design should also be included for any third-party entropy sources that are included in the product.

The documentation shall describe how unprocessed (raw) data was obtained for the analysis. This description shall be sufficiently detailed to explain at what point in the entropy source model the data was collected and what effects, if any, the process of data collection had on the overall entropy generation rate. The documentation should walk through the entropy source design indicating where the entropy comes from, where the entropy output is passed next, any post-processing of the raw outputs (hash, XOR, etc.), if/where it is stored, and finally, how it is output from the entropy source. Any conditions placed on the process (e.g., blocking) should also be described in the entropy source design. Diagrams and examples are encouraged.

This design must also include a description of the content of the security boundary of the entropy source and a description of how the security boundary ensures that an adversary outside the boundary cannot affect the entropy rate.

If implemented, the design description shall include a description of how third-party applications can add entropy to the RBG. A description of any RBG state saving between power-off and power-on shall be included.

D.2 Entropy Justification

There should be a technical argument for where the unpredictability in the source comes from and why there is confidence in the entropy source delivering sufficient entropy for the uses made of the RBG output (by this particular TOE). This argument will include a description of the expected min-entropy rate (i.e. the minimum entropy (in bits) per bit or byte of source data) and explain that sufficient entropy is going into the TOE randomizer seeding process. This discussion will be part of a justification for why the entropy source can be relied upon to produce bits with entropy.

The amount of information necessary to justify the expected min-entropy rate depends on the type of entropy source included in the product.

For developer provided entropy sources, in order to justify the min-entropy rate, it is expected that a large number of raw source bits will be collected, statistical tests will be performed, and the min-entropy rate determined from the statistical tests. While no particular statistical tests are required at this time, it is expected that some testing is necessary in order to determine the amount of min-entropy in each output.

For third party provided entropy sources, in which the TOE vendor has limited access to the design and raw entropy data of the source, the documentation will indicate an estimate of the amount of min-entropy obtained from this third-party source. It is acceptable for the vendor to “assume” an amount of min-entropy, however, this assumption must be clearly stated in the documentation provided. In particular, the min-entropy estimate must be specified and the assumption included in the ST.

Regardless of type of entropy source, the justification will also include how the DRBG is initialized with the entropy stated in the ST, for example by verifying that the min-entropy rate is multiplied by the amount of source data used to seed the DRBG or that the rate of entropy expected based on the amount of source data is explicitly stated and compared to the statistical rate. If the amount of source data used to seed the DRBG is not clear or the calculated rate is not explicitly related to the seed, the documentation will not be considered complete.

The entropy justification shall not include any data added from any third-party application or from any state saving between restarts.

D.3 Operating Conditions

The entropy rate may be affected by conditions outside the control of the entropy source itself. For example, voltage, frequency, temperature, and elapsed time after power-on are just a few of the factors that may affect the operation of the entropy source. As such,

documentation will also include the range of operating conditions under which the entropy source is expected to generate random data. It will clearly describe the measures that have been taken in the system design to ensure the entropy source continues to operate under those conditions. Similarly, documentation shall describe the conditions under which the entropy source is known to malfunction or become inconsistent. Methods used to detect failure or degradation of the source shall be included.

D.4 Health Testing

More specifically, all entropy source health tests and their rationale will be documented. This will include a description of the health tests, the rate and conditions under which each health test is performed (e.g., at startup, continuously, or on-demand), the expected results for each health test, and rationale indicating why each test is believed to be appropriate for detecting one or more failures in the entropy source.

Appendix E - Application Software Equivalency Guidelines

E.1 Introduction

The purpose of equivalence in PP-based evaluations is to find a balance between evaluation rigor and commercial practicability—to ensure that evaluations meet customer expectations while recognizing that there is little to be gained from requiring that every variation in a product or platform be fully tested. If a product is found to be compliant with a PP on one platform, then all equivalent products on equivalent platforms are also considered to be compliant with the PP.

A Vendor can make a claim of equivalence if the Vendor believes that a particular instance of their Product implements PP-specified security functionality in a way equivalent to the implementation of the same functionality on another instance of their Product on which the functionality was tested. The Product instances can differ in version number or feature level (model), or the instances may run on different platforms. Equivalency can be used to reduce the testing required across claimed evaluated configurations. It can also be used during Assurance Maintenance to reduce testing needed to add more evaluated configurations to a certification.

These equivalency guidelines do not replace Assurance Maintenance requirements or scheme-specific policy requirements for CAVP certificates. Nor may equivalency be used to leverage evaluations with expired certifications.

These Equivalency Guidelines represent a shift from complete testing of all product instances to more of a risk-based approach. Rather than require that every combination of product and platform be tested, these guidelines support an approach that recognizes that products are being used in a variety of environments—and often in cloud environments over where the vendor (and sometimes the customer) have little or no control over the underlying hardware. Developers should be responsible for the security functionality of their applications on the platforms they are developed for—whether that is an operating system, a virtual machine, or a software-based execution environment such as a container. But those platforms may themselves run within other environments—virtual machines or operating systems—that completely abstract away the underlying hardware from the application. The developer should not be held accountable for security functionality that is implemented by platform layers that are abstracted away. The implication is that not all security functionality will necessarily be tested for all platform layers down to the hardware for all evaluated configurations—especially for applications developed for software-based execution environments such as containers. For these cases, the balancing of evaluation rigor and commercial practicability tips in favor of practicability. Note that this does not affect the requirement that at least one product instance be fully tested on at least one platform with cryptography mapped to a CAVP certificate.

Equivalency has two aspects:

1. **Product Equivalence:** Products may be considered equivalent if there are no differences between Product Models and Product Versions with respect to PP-specified security functionality.
2. **Platform Equivalence:** Platforms may be considered equivalent if there are no significant differences in the services they provide to the Product—or in the way the platforms provide those services—with respect to PP-specified security functionality.

The equivalency determination is made in accordance with these guidelines by the Validator and Scheme using information provided by the Evaluator/Vendor.

E.2 Approach to Equivalency Analysis

There are two scenarios for performing equivalency analysis. One is when a product has been certified and the vendor wants to show that a later product should be considered certified due to equivalence with the earlier product. The other is when multiple product variants are going through evaluation together and the vendor would like to reduce the amount of testing that must be done. The basic rules for determining equivalence are the same in both cases. But there is one additional consideration that applies to equivalence with previously certified products. That is, the product with which equivalence is being claimed must have a valid certification in accordance with scheme rules and the Assurance Maintenance process must be followed. If a product's certification has expired, then equivalence cannot be claimed with that product.

When performing equivalency analysis, the Evaluator/Vendor should first use the factors and guidelines for Product Model equivalence to determine the set of Product Models to be evaluated. In general, Product models that do not differ in PP-specified security functionality are considered equivalent for purposes of evaluation against the this PP.

If multiple revision levels of Product Models are to be evaluated—or to determine whether a revision of an evaluated product needs re-evaluation—the Evaluator/Vendor and Validator should use the factors and guidelines for Product Version equivalence to analyze whether Product Versions are equivalent.

Having determined the set of Product Models and Versions to be evaluated, the next step is to determine the set of Platforms that the Products must be tested on.

Each non-equivalent Product for which compliance is claimed must be fully tested on each non-equivalent platform for which compliance is claimed. For non-equivalent Products on equivalent platforms, only the differences that affect PP-specified security functionality must be tested for each product.

“Differences in PP-Specified Security Functionality” Defined

If PP-specified security functionality is implemented by the TOE, then differences in the actual implementation between versions or product models break equivalence for that feature. Likewise, if the TOE implements the functionality in one version or model and the functionality is implemented by the platform in another version or model, then equivalence is broken. If the functionality is implemented by the platform in multiple models or versions on equivalent platforms, then the functionality is considered different if the product invokes the platform differently to perform the function.

E.3 Specific Guidance for Determining Product Model Equivalence

Product Model equivalence attempts to determine whether different feature levels of the same product across a product line are equivalent for purposes of PP testing. For example, if a product has a “basic” edition and an “enterprise” edition, is it necessary to test both models? Or does testing one model provide sufficient assurance that both models are compliant?

Product models are considered equivalent if there are no differences that affect PP-specified security functionality—as indicated in Table 1.

Factor	Same/Different	Guidance
PP-Specified Functionality	Same	If the differences between Models affect only non-PP-specified functionality, then the Models are equivalent.
	Different	If PP-specified security functionality is affected by the differences between Models, then the Models are not equivalent and must be tested separately. It is necessary only to test the functionality affected by the software differences. If only differences are tested, then the differences must be enumerated, and for each difference the Vendor must provide an explanation of why each difference does or does not affect PP-specified functionality. If the Product Models are separately tested fully, then there is no need to document the differences.

Table 1. Determining Product Model Equivalence

E.4 Specific Guidance for Determining Product Version Equivalence

In cases of version equivalence, differences are expressed in terms of changes implemented in revisions of an evaluated Product. In general, versions are equivalent if the changes have no effect on any security-relevant claims about the TOE or assurance evidence. Non-security-relevant changes to TOE functionality or the addition of non-security-relevant functionality does not affect equivalence.

Factor	Same/Different	Guidance
Product Models	Different	Versions of different Product Models are not equivalent unless the Models are equivalent as defined in Section 3.
PP-Specified Functionality	Same	If the differences affect only non-PP-specified functionality, then the Versions are equivalent.
	Different	If PP-specified security functionality is affected by the differences, then the Versions are not considered equivalent and must be tested separately. It is necessary only to test the functionality affected by the changes. If only the differences are tested, then for each difference the Vendor must provide an explanation of why the difference does or does not affect PP-specified functionality. If the Product Versions are separately tested fully, then there is no need to document the differences.

Table 2. Factors for Determining Product Version Equivalence

E.5 Specific Guidance for Determining Platform Equivalence

Platform equivalence is used to determine the platforms that equivalent versions of a Product must be tested on. Platform equivalence analysis done for one software application cannot be applied to another software application. Platform equivalence is not general—it is with respect to a particular application.

Product Equivalency analysis must already have been done and Products have been determined to be equivalent.

The platform can be hardware or virtual hardware, an operating system or similar entity, or a software execution environment such as a container. For purposes of determining equivalence for software applications, we address each type of platform separately. In general, platform equivalence is based on differences in the interfaces between the **TQE** and Platform that are relevant to the implementation of **PP**-specified security functionality.

E.5.1 Platform Equivalence—Hardware/Virtual Hardware Platforms

If an application runs directly on hardware without an operating system—or directly on virtualized hardware without an operating system—then platform equivalence is based on processor architecture and instruction sets. In the case of virtualized hardware, it is the virtualized processor and architecture that are presented to the application that matters—not the physical hardware.

Platforms with different processor architectures and instruction sets are not equivalent. This is not likely to be an issue for equivalency analysis for applications since there is likely to be a different version of the application for different hardware environments. Equivalency analysis becomes important when comparing processors with the same architecture. Processors with the same architecture that have instruction sets that are subsets or supersets of each other are not disqualified from being equivalent for purposes of an App evaluation. If the application takes the same code paths when executing **PP**-specified security functionality on different processors of the same family, then the processors can be considered equivalent with respect to that application. For example, if an application follows one code path on platforms that support the **AES-NI** instruction and another on platforms that do not, then those two platforms are not equivalent with respect to that application functionality. But if the application follows the same code path whether or not the platform supports **AES-NI**, then the platforms are equivalent with respect to that functionality.

The platforms are equivalent with respect to the application if the platforms are equivalent with respect to all **PP**-specified security functionality.

Factor	Same/Different/None	Guidance
Platform Architectures	Different	Platforms that present different processor architectures and instruction sets to the application are not equivalent.
PP-Specified Functionality	Same	For platforms with the same processor architecture, the platforms are equivalent with respect to the application if execution of all PP -specified security functionality follows the same code path on both platforms.

Table 3. Factors for Determining Hardware/Virtual Hardware Platform Equivalence

E.5.2 Platform Equivalence—OS Platforms

For traditional applications that are built for and run on operating systems, platform equivalence is determined by the interfaces between the application and the operating system that are relevant to **PP**-specified security functionality. Generally, these are the processor interface, device interfaces, and **QS** APIs. The following factors are applied in order:

Factor	Same/Different/None	Guidance
Platform Architectures	Different	Platforms that run on different processor architectures and instruction sets are not equivalent.
Platform Vendors	Different	Platforms from different vendors are not equivalent.
Platform Versions	Different	Platforms from the same vendor with different major version numbers are not equivalent.
Platform Interfaces	Different	Platforms from the same vendor and major version are not equivalent if there are differences in device interfaces and QS APIs that are relevant to the way the platform provides PP -specified security functionality to the application.
Platform Interfaces	Same	Platforms from the same vendor and major version are equivalent if there are no differences in device interfaces and QS APIs that are relevant to the way the platform provides PP -specified security functionality to the application, or if the Platform does not provide such functionality to the application.

Table 4. Factors for Determining QS/VS Platform Equivalence

E.5.3 Software-based Execution Environment Platform Equivalence

If an Application is built for and runs in a non-OS software-based execution environment, such as a Container or Java Runtime, then the below criteria must be used to determine platform equivalence. The key point is that the underlying hardware (virtual or physical) and **QS** is

not relevant to platform equivalence. This allows applications to be tested and run on software-based execution environments on any hardware—as in cloud deployments.

Factor	Same/Different/None	Guidance
Platform Type/Vendor	Different	Software-based execution environments that are substantially different or come from different vendors are not equivalent. For example, a Java virtual machine is not the same as a container. A Docker container is not the same as a CoreOS container.
Platform Versions	Different	Execution environments that are otherwise equivalent are not equivalent if they have different major version numbers.
PP-Specified Security Functionality	Same	All other things being equal, execution environments are equivalent if there is no significant difference in the interfaces through which the environments provide PP-specified security functionality to applications.

Table 5. Factors for Software-based Execution Environment Platform Equivalence

E.6 Level of Specificity for Tested Configurations and Claimed Equivalent Configurations

In order to make equivalency determinations, the vendor and evaluator must agree on the equivalency claims. They must then provide the scheme with sufficient information about the **TQE** instances and platforms that were evaluated, and the **TQE** instances and platforms that are claimed to be equivalent.

The **ST** must describe all configurations evaluated down to processor manufacturer, model number, and microarchitecture version.

The information regarding claimed equivalent configurations depends on the platform that the application was developed for and runs on.

Bare-Metal Applications

For applications that run without an operating system on bare-metal or virtual bare-metal, the claimed configuration must describe the platform down to the specific processor manufacturer, model number, and microarchitecture version. The Vendor must describe the differences in the **TQE** with respect to PP-specified security functionality and how the **TQE** functions differently to leverage platform differences (e.g., instruction set extensions) in the tested configuration versus the claimed equivalent configuration.

Traditional Applications

For applications that run with an operating system as their immediate platform, the claimed configuration must describe the platform down to the specific operating system version. If the platform is a virtualization system, then the claimed configuration must describe the platform down to the specific virtualization system version. The Vendor must describe the differences in the **TQE** with respect to PP-specified security functionality and how the **TQE** functions differently to leverage platform differences in the tested configuration versus the claimed equivalent configuration. Relevant platform differences could include instruction sets, device interfaces, and **QS** APIs invoked by the **TQE** to implement PP-specified security functionality.

Software-Based Execution Environments

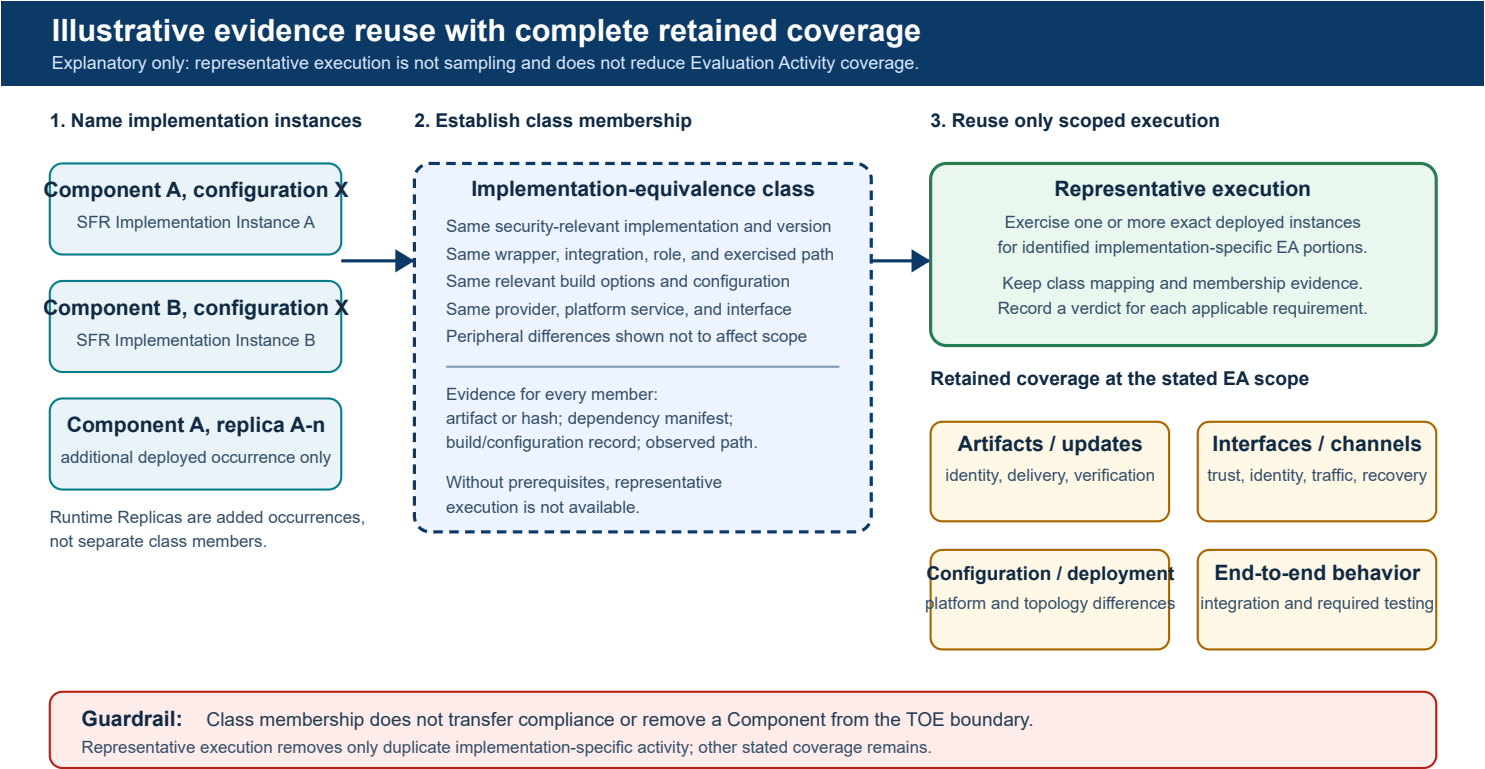
For applications that run in a software-based execution environment such as a Java virtual machine or a Container, then the claimed configuration must describe the platform down to the specific version of the software execution environment. The Vendor must describe the differences in the **TQE** with respect to PP-specified security functionality and how the **TQE** functions differently to leverage platform differences in the tested configuration versus the claimed equivalent configuration.

Appendix F - Component Implementation Equivalence Within a Distributed TOE

F.1 Purpose and Relationship to Product Equivalence

Component implementation equivalence is an evidence-reuse mechanism for named SFR Implementation Instances used by one or more TOE Components and identified security-relevant configurations within one distributed TOE. It is distinct from Product Equivalence and Platform Equivalence, which compare product models, product versions, or evaluated configurations. Interfaces and communication relationships are not class members; they remain coverage targets and retain every interface-specific, channel-specific, and integration activity required by the applicable Evaluation Activities. Component implementation equivalence does not transfer compliance from one TOE Component or identified configuration to another. Every TOE Component remains subject to every SFR allocated to it, and every TOE Component Instance shall exhibit the claimed behavior of its TOE Component in its identified configuration. A Runtime Replica is not a separate SFR Implementation Instance or implementation-equivalence class member solely because it is an additional deployed occurrence.

The ST need not repeat identical completed SFR text or identical TSS descriptions for every TOE Component. The ST shall provide concise traceability from each TOE Component and identified security-relevant configuration, where applicable, to the applicable SFR iteration and, where evidence reuse is claimed, to a class identifier and common TSS description. The detailed rationale and the class-to-Evaluation-Activity mapping may be provided separately from the ST. A class definition may list multiple SFR iterations and implementation-specific portions of Evaluation Activities when the member SFR Implementation Instances and equivalence basis are identical for the entire listed scope.



security-relevant implementation, version, provider, wrapper or integration behavior, build options, implementation configuration and values that affect the reused activities, role, exercised code path, and relevant platform services and interfaces are prerequisites for class membership. Only peripheral differences outside that implementation may be addressed by a rationale that they cannot affect the scoped results. Channel-specific certificates, keys, trust-anchor values, endpoint identities, addresses, and similar deployed values may differ when they do not alter the reused implementation behavior and are covered by the retained interface- or channel-specific activities. When the prerequisites are not established, the affected **SER** Implementation Instances may not share representative execution for that scope.

Use of a common library name, base image, runtime, source repository, or build pipeline is relevant evidence but is not sufficient by itself. Component-specific calling code or configuration can change security behavior even when the underlying library is shared. Likewise, an operational-environment service such as a service mesh or ingress controller is not a shared **TOE** implementation unless it is explicitly included in the **TOE** boundary.

For **TLS** and similar trusted-channel functionality, the **SER** Implementation Instances in a class shall have the same client or server role, **TLS** versus mutual-TLS behavior, protocol implementation and version, wrapper or integration behavior, supported protocol versions and algorithms, cryptographic provider, certificate validation and reference-identifier processing, revocation behavior, trust and credential processing and source mechanism, immediate platform services and interfaces relevant to the reused protocol tests, and implementation configuration relevant to those tests. Channel-specific certificates, keys, trust-anchor values, endpoint identities, addresses, and similar deployed values may differ because they remain subject to the applicable interface- and channel-specific checks. Different interruption or recovery behavior does not necessarily change the protocol implementation class, but each distinct channel remains subject to the applicable interruption and recovery tests.

Class membership shall be re-established when a member's security-relevant implementation version, provider, wrapper or integration behavior, relevant build options, relevant implementation configuration, relevant platform service or interface, or exercised code path changes. A change outside the scoped implementation does not automatically invalidate the class, but the rationale shall be updated when the change is reasonably capable of affecting a reused result.

F.3 Representative Execution and Complete Coverage

The evaluator selects one or more representative class-member **SER** Implementation Instances for each substantiated class and, for each, an exact deployed **TOE** Component Instance through which it will be exercised. The evaluator performs the explicitly identified implementation-specific portions of the applicable Evaluation Activities using deployed **TOE** Component Instances that collectively expose all relevant behavior. The evaluator verifies membership of every other class member using objective implementation, dependency, build, and configuration evidence. This is representative execution with complete class-membership verification; it is not sampling.

Representative execution eliminates only duplicate execution of the same implementation-specific activity across mapped class members. It does not reduce any test count, parameter combination, supported algorithm, protocol role, configuration, certificate case, negative test, interface, channel, or communication-relationship coverage required by the originating Evaluation Activity. This rule also applies to Evaluation Activities incorporated from **PP**-Modules and Functional Packages.

Artifact-specific examination, tests of distinct interfaces, roles, configurations, component identities, or communication relationships, activities that expressly require every member to be examined or tested, and end-to-end testing of the distributed **TOE** remain applicable at their stated scope. A single execution may support more than one mapped **SER** Implementation Instance, **SER**, or Evaluation Activity only when it demonstrates every applicable objective and the evaluation report preserves traceability and a separate verdict for each.

Appendix G - Acronyms

Table 4: Acronyms

Acronym	Meaning
ADB	Android Debug Bridge
AES	Advanced Encryption Standard
API	Application Programming Interface
APK	Android Application Package
app	Application
APPX	Windows Universal Application Package
ASLR	Address Space Layout Randomization
Base-PP	Base Protection Profile
BIOS	Basic Input/Output System
CC	Common Criteria
CEM	Common Evaluation Methodology
CMC	Certificate Management over CMS
CMS	Cryptographic Message Syntax
cPP	Collaborative Protection Profile
DEP	Data Execution Prevention
DES	Data Encryption Standard
DHE	Diffie-Hellman Ephemeral
DMG	Apple Disk Image
DNS	Domain Name System
DPAPI	Data Protection Application Programming Interface
DRBG	Deterministic Random Bit Generator
DSS	Digital Signature Standard
DT	Date/Time Vector
DTLS	Datagram Transport Layer Security
EAP	Extensible Authentication Protocol
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
ECDSA	Elliptic Curve Digital Signature Algorithm
ELF	Executable and Linkable Format
EMET	Enhanced Mitigation Experience Toolkit

EP	Extended Package
EST	Enrollment over Secure Transport
FIPS	Federal Information Processing Standards
FP	Functional Package
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IANA	Internet Assigned Number Authority
IEC	International Electrotechnical Commission
IETF	Internet Engineering Task Force
IP	Internet Protocol
IPA	iOS Package archive
IR	Intermediate Integer
ISO	International Organization for Standardization
IT	Information Technology
ITSEF	Information Technology Security Evaluation Facility
JNI	Java Native Interface
LDAP	Lightweight Directory Access Protocol
MIME	Multi-purpose Internet Mail Extensions
MPKG	Meta Package
MSI	Microsoft Installer
NFC	Near Field Communication
NIAP	National Information Assurance Partnership
NIST	National Institute of Standards and Technology
OCSP	Online Certificate Status Protocol
OE	Operational Environment
OID	Object Identifier
OMB	Office of Management and Budget
OS	Operating System
PDF	Portable Document Format
PE	Portable Executable
PID	Process Identifier
PII	Personally Identifiable Information
PKG	Package file

PKI	Public Key Infrastructure
PP	Protection Profile
PP-Configuration	Protection Profile Configuration
PP-Module	Protection Profile Module
RBG	Random Bit Generator
REC	Request for Comment
RNGVS	Random Number Generator Validation System
S/MIME	Secure/Multi-purpose Internet Mail Extensions
SAN	Subject Alternative Name
SAR	Security Assurance Requirement
SE	Security Enhancements
SER	Security Functional Requirement
SHA	Secure Hash Algorithm
SIP	Session Initiation Protocol
SP	Special Publication
SSH	Secure Shell
ST	Security Target
SWID	Software Identification
TLS	Transport Layer Security
TOE	Target of Evaluation
TSF	TOE Security Functionality
TSEI	TSF Interface
TSS	TOE Summary Specification
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
USB	Universal Serial Bus
XCCDF	eXtensible Configuration Checklist Description Format
XOR	Exclusive Or

Appendix H - Bibliography

Table 5: Bibliography

Identifier	Title
[CC]	Common Criteria for Information Technology Security Evaluation - - Part 1: Introduction and general model, CCMB-2022-11-001, CC:2022, Revision 1, November 2022. - Part 2: Security functional requirements, CCMB-2022-11-002, CC:2022, Revision 1, November 2022. - Part 3: Security assurance requirements, CCMB-2022-11-003, CC:2022, Revision 1, November 2022. - Part 4: Framework for the specification of evaluation methods and activities, CCMB-2022-11-004, CC:2022, Revision 1, November 2022. - Part 5: Pre-defined packages of security requirements, CCMB-2022-11-005, CC:2022, Revision 1, November 2022.
[CEM]	Common Methodology for Information Technology Security Evaluation - Evaluation methodology, CCMB-2022-11-006, CC:2022, Revision 1, November 2022.
[OMB]	Reporting Incidents Involving Personally Identifiable Information and Incorporating the Cost for Security in Agency Information Technology Investments, OMB M-06-19, July 12, 2006.
[SHAKE]	SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions, NIST FIPS PUB 202, August 2015.
[TD0994]	NIAP Technical Decision TD0994: Addition of FCS_COP.1/XOF to PP_APP_V2.0, May 13, 2026.